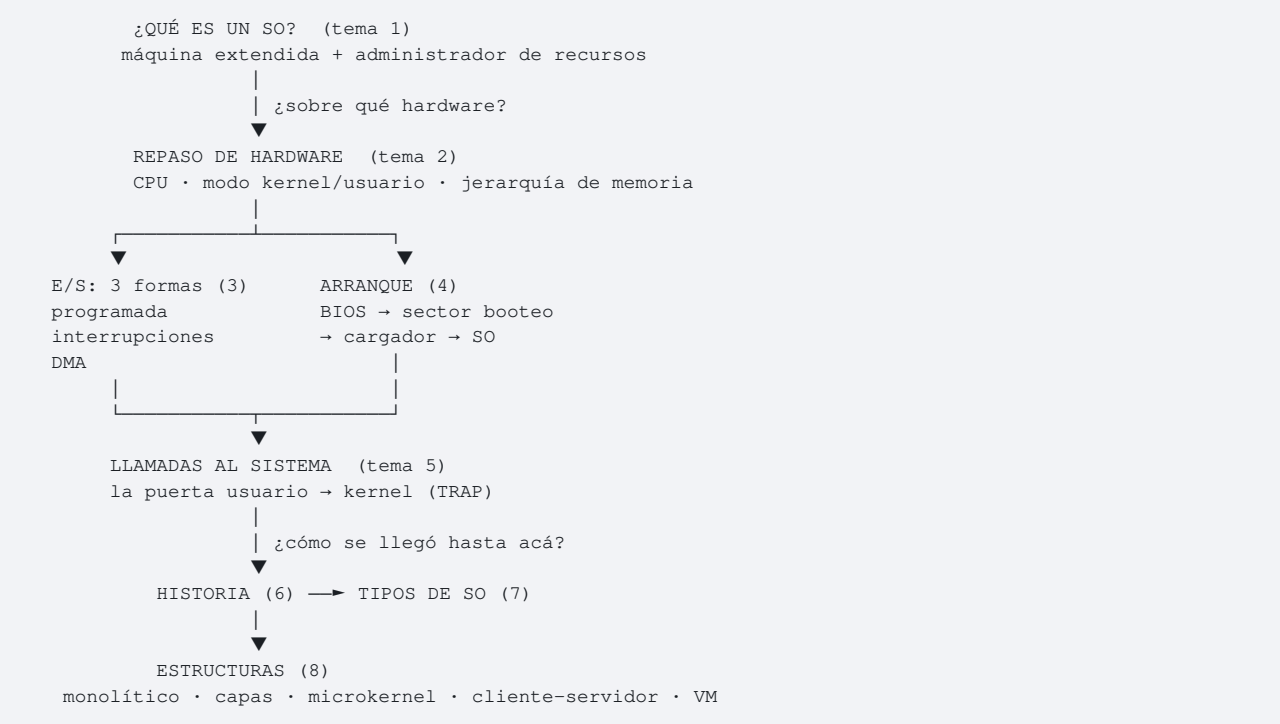
Unidad 1 — Introducción, hardware y estructura de los SO

Sistemas Operativos — UTN FRM — Repaso de la clase completa en el chat: 9 temas, correcciones y cierre

Cómo leer este repaso. Es la clase tal como se dio, tema por tema, con los mismos diagramas y las re-explicaciones que pediste. Al final de cada tema están las **preguntas de control** y su **corrección**, donde quedaron marcados los errores que conviene no repetir. La referencia al Tanenbaum está al final de cada tema. El resumen de todo está en el **Cierre de la Unidad 1**, en las últimas páginas.

Mapa de la unidad

Antes de arrancar, mirá cómo se conectan los temas, porque no es una lista suelta: es una cadena causal.



La idea que atraviesa todo: el SO existe para esconder un hardware incómodo y para repartirlo entre varios que lo quieren al mismo tiempo. Temas 2 a 4 son *el hardware que hay que esconder*. Tema 5 es *la puerta por donde se pide*. Temas 6 a 8 son *cómo se fue organizando por dentro*.

Son 8 temas. Te los doy de a uno.

Tema 1 — ¿Qué es un sistema operativo?

La intuición

Pensá en un consorcio de un edificio.

El encargado hace **dos cosas que no tienen nada que ver entre sí**:

- Te **simplifica la vida**: vos no sabés dónde está la bomba de agua ni cómo se purga la caldera. Abrís la canilla y sale agua. Él te vende una abstracción cómoda.
- Te **reparte lo escaso**: hay una sola cochera para visitas y ocho departamentos. Alguien tiene que decidir turnos.

El sistema operativo es eso. Y Tanenbaum es muy explícito en que son **dos funciones básicas que no están**

relacionadas — por eso cuesta definirlo.

El concepto formal

Función 1 — El SO como máquina extendida.

La arquitectura a nivel de lenguaje máquina es *primitiva y compleja de programar*. La **abstracción** es la clave para lidiar con esa complejidad. La abstracción que todo el mundo entiende es el **archivo**.

Mirá la diferencia concreta:

```
LEER UN ARCHIVO                  LEER UN ARCHIVO
SIN sistema operativo            CON sistema operativo
─────────────────────            ──────────────────────

1. encender motor del disco
2. esperar velocidad
3. mover brazo al cilindro       ┌──────────────────────┐
4. esperar que pase el sector    │ read(fd, buf, nbytes)│
5. leer los bits                 └──────────────────────┘
6. verificar código de error
7. apagar el motor
```

> "El trabajo del sistema operativo es **crear buenas abstracciones** para después implementar y administrar los objetos abstractos entonces creados."

Función 2 — El SO como administrador de recursos.

Proporciona una **asignación ordenada y controlada** de procesadores, memorias y dispositivos de E/S entre los programas que compiten por ellos. Y eso lo hace **multiplexando** de dos formas:

	Multiplexaje en el **TIEMPO**	Multiplexaje en el **ESPACIO**
Idea	Toman **turnos**	Cada uno recibe **una parte**
Esquema	`A──B──A──C──A──B──►`	`┌─A─┬──B──┬─C─┐`
Recursos	CPU, impresora	Memoria principal, disco
Problema que genera	¿Quién va primero? → **planificación** (U2)	¿Y si uno pisa al otro? → **protección** (U3)

Fijate que esa última fila es el programa entero de la materia. Las unidades que siguen son, básicamente, resolver esas dos preguntas.

Ejemplo concreto

Abrís el navegador y Spotify al mismo tiempo en una máquina con 1 CPU:

- **Tiempo:** la CPU va alternando entre los dos, decenas de veces por segundo. Vos percibís que suenan y navegás "a la vez", pero nunca están los dos ejecutando en el mismo instante.
- **Espacio:** la RAM está partida. El navegador ocupa su pedazo, Spotify el suyo, y ninguno puede leer el del otro.

Referencia

(Tanenbaum, cap. 1.1, págs. impresas **3 a 7** → PDF **33 a 37**). También en `fuentes/resumen_catedra_u01.pdf`, sección 1.1.

Preguntas de control

1. Un disco duro, ¿se multiplexa en el tiempo o en el espacio? ¿Y por qué podría discutirse que es en los dos?

2. Te digo: *"el sistema operativo es el programa que te deja usar la computadora más fácil"*. Según Tanenbaum, esa definición está **incompleta**. ¿Qué le falta?

Respondeme y seguimos con el tema 2.

Corrección

Pregunta 1 — Bien la primera parte. El disco se multiplexa **en el espacio**: cada usuario recibe *una parte* (sus bloques), no un turno.

Tu justificación del "en los dos" apunta a algo real, pero la razón precisa es otra. Mirá:

```
EL DISCO SON DOS RECURSOS DISTINTOS

  Superficie magnética  →  hay MUCHA  →  se reparte  →  ESPACIO
  (los bloques)                            en partes

  El brazo / cabezal    →  hay UNO    →  se turna    →  TIEMPO

  ┌──────────────────────────────┐
  │ bloques de Ana | de Juan |..|   ← espacio: cada uno su pedazo
  └──────────────────────────────┘

          ▲
          | un solo brazo          ← tiempo: Ana, después Juan,
       ═══════                        después Ana...
```

Vos pensaste "lo del espacio depende de lo del tiempo" — y es cierto, pero el motivo de fondo es que **hay dos recursos físicos adentro del mismo aparato**, y cada uno se multiplexa distinto. Esa diferencia la vas a usar en serio en la Unidad 5, cuando veas los algoritmos de planificación de disco (FCFS, SSTF, SCAN): ahí lo único que se discute es cómo repartir el brazo **en el tiempo**.

Pregunta 2 — Correcta. Le falta administrador de recursos, y lo dijiste. Un solo retoque de vocabulario para el examen: la frase del libro es *"proporcionar una **asignación ordenada y controlada** de los procesadores, memorias y dispositivos de E/S entre los programas que compiten por ellos"*. Las palabras *ordenada y controlada* valen puntos.

Vamos al tema 2.

Tema 2 — Repaso del hardware

La intuición

Para entender al SO hay que ver primero con qué está peleando. Tres cosas: una CPU que ejecuta una instrucción por vez, una memoria que siempre es demasiado lenta o demasiado chica, y un montón de dispositivos lentísimos.

2.1 La CPU y sus registros

El ciclo básico de toda CPU, textual del libro: **obtener** la instrucción de memoria, **decodificarla** para determinar su tipo y operandos, **ejecutarla**, y volver a empezar.

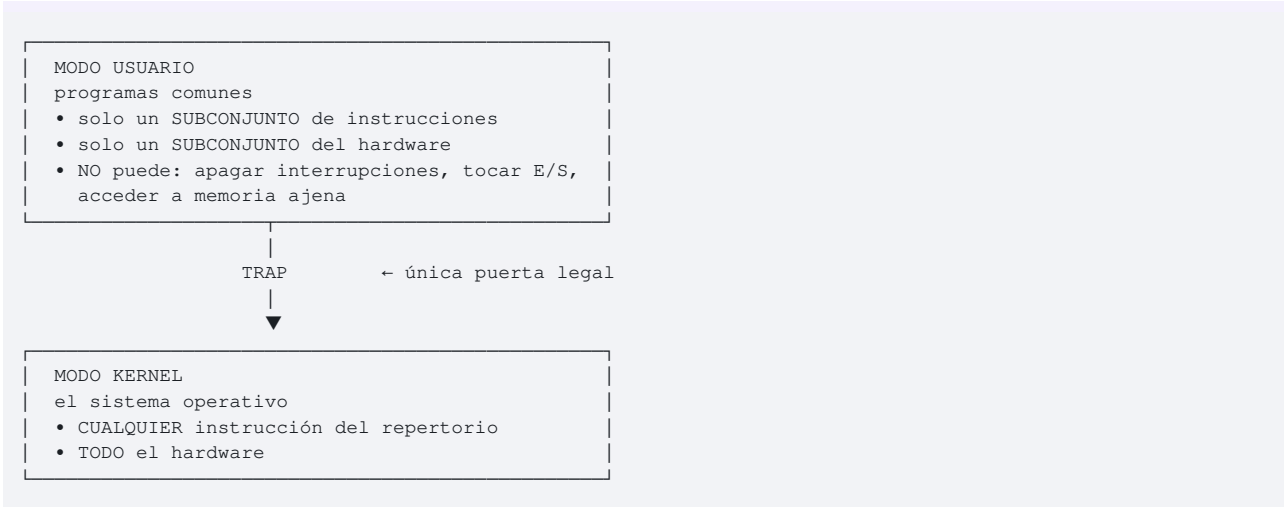
Tres registros especiales que te van a pedir con nombre y apellido:

Registro	Qué contiene	Por qué le importa al SO
Contador de programa (PC)	Dirección de la **siguiente** instrucción a obtener	Para suspender y reanudar un proceso
Apuntador de pila (SP)	Tope de la pila actual: valores de los procedimientos llamados y no retornados	Cada proceso tiene su propia pila
PSW (palabra de estado del programa)	Bits de código de condición **y el bit de modo**	Acá vive la protección

El sistema operativo **debe estar al tanto de todos los registros**, porque cuando cambia de proceso tiene que guardarlos y restaurarlos todos. Eso se llama cambio de contexto y lo vemos en la Unidad 2.

2.2 El tema que SÍ o SÍ te toman: modo kernel y modo usuario

Un bit del PSW decide en qué modo está la CPU.

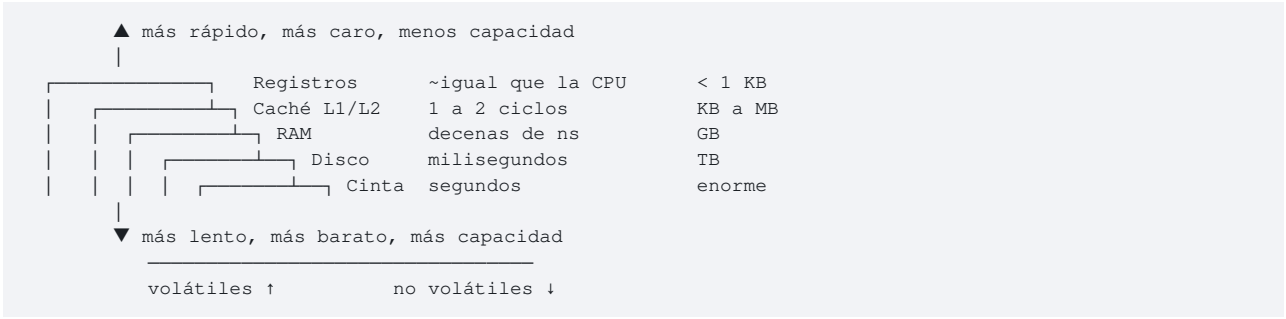


Por qué esto es el corazón de todo: sin estos dos modos no habría protección, no habría multiprogramación segura, y cualquier programa podría formatearte el disco. Toda la materia se apoya acá.

La frase para el examen: *"Para obtener servicios del SO, un programa de usuario debe lanzar una **llamada al sistema**, que se **atrapa** en el kernel. La instrucción **TRAP** cambia del modo usuario al modo kernel e inicia el sistema operativo."*

2.3 La jerarquía de memoria

La regla: **más rápido** ⇒ **más caro** ⇒ **más chico**.



Cómo funciona la caché: la memoria principal se divide en **líneas de caché**. Las líneas más usadas se mantienen en una caché rápida dentro o muy cerca de la CPU. Cuando el programa pide una palabra, el hardware revisa si la línea está ahí:

- **Acierto de caché** → se resuelve ahí, **no se envía petición por el bus** a la memoria principal.
- **Fallo** → hay que ir a la RAM y pagar el precio completo.

L1 está siempre dentro de la CPU y se accede **sin retraso**; L2 tiene varios MB y cuesta **uno o dos ciclos de reloj**.

Ejemplo resuelto — ¿cuánto sirve realmente la caché?

Caché L1 = **1 ns** · RAM = **10 ns** · aciertos = **95 %**

```
Paso 1   95% de las veces:  está en caché       →  1 ns
Paso 2    5% de las veces:  fallo               →  1 ns (mirar) + 10 ns (RAM) = 11 ns
Paso 3   promedio ponderado:
         Tm = 0,95 × 1  +  0,05 × 11
            = 0,95      +  0,55
            = 1,50 ns
Paso 4   sin caché sería 10 ns  →  10 / 1,50 = 6,67 veces más rápido
```

Lo fuerte: una caché **ínfima** comparada con la RAM te da casi 7× de mejora. Eso funciona por la **localidad de referencia**, y es exactamente la misma idea que vas a reencontrar en la Unidad 3 con la memoria virtual (la RAM como caché del disco) y en la Unidad 4 con la caché de bloques del sistema de archivos. Es el mismo truco tres veces.

2.4 El resto, más rápido

- **Memoria no volátil:** ROM (de fábrica, no se modifica) · EEPROM y flash (sí se reescriben) · **CMOS** (volátil, con batería: guarda fecha, hora y config del BIOS).
- **Disco:** platos que giran a 5400, 7200 o 10800 rpm. **Pista** = región anular que lee una cabeza. **Cilindro** = todas las pistas para una misma posición del brazo. *(Guardá estos dos términos: en U5 hacés cuentas con ellos.)*
- **MMU** (Memory Management Unit): la parte de la CPU que traduce la dirección que genera el programa en la dirección física real de la RAM. Es el hardware de la memoria virtual (U3).
- **Dispositivos de E/S:** siempre son **dos partes**, el dispositivo + su **controlador** (chip). El software que le habla al controlador es el **driver**, y tiene que correr en modo kernel.
- **Buses:** ISA (viejo) → PCI · USB para dispositivos lentos (teclado, mouse) · SCSI para dispositivos veloces.

Referencia

(Tanenbaum, cap. 1.3, págs. impresas **19 a 33** → PDF **49 a 63**).

Preguntas de control

1. Un programa de usuario ejecuta una instrucción que apaga las interrupciones de la máquina. ¿Qué pasa y por qué? ¿Qué tendría que haber hecho si de verdad necesitaba ese servicio?

2. Misma máquina del ejemplo (L1 = 1 ns, RAM = 10 ns), pero el programa tiene **mala localidad** y la tasa de aciertos cae al **50 %**. Calculá el tiempo medio de acceso. ¿Vale la pena la caché todavía?

Corrección

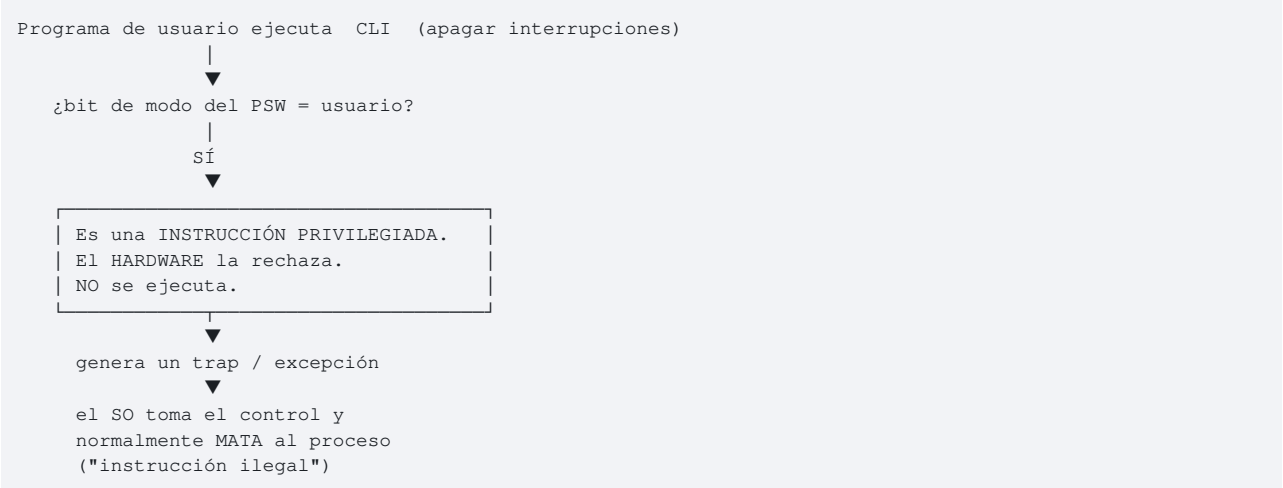
Pregunta 2 — Perfecto el cálculo. Tm = 0,5 × 1 + 0,5 × 11 = **6,00 ns**, y 10/6 = **1,67×**. Clavado.

Y tu conclusión es la correcta: la caché **vale lo que vale la localidad del programa**. Con 95 % de aciertos da 6,67×; con 50 % da 1,67×. El hardware es el mismo — lo que cambió es el programa. Por eso la localidad de referencia es un concepto de software, no de hardware, y por eso en la Unidad 3 vas a ver programas que generan cuatro veces más fallos de página solo por recorrer una matriz por columnas en vez de por filas.

Pregunta 1 — Acá hay un error que te conviene corregir ahora, porque es muy tomado.

Tu segunda mitad está impecable (TRAP → modo kernel → servicio → protección). El problema está en la primera: dijiste que *"el sistema operativo deja de responder"*.

Eso es lo que pasaría **si la instrucción llegara a ejecutarse**. Y es justamente lo que el hardware **impide**:



La diferencia es clave: **no es que el SO se cuelgue y después reaccione. Es que la instrucción nunca llega a ejecutarse.** La protección no es un castigo posterior, es una barrera previa.

Dos términos para el examen:

- **Instrucción privilegiada**: la que solo corre en modo kernel. Apagar interrupciones, tocar registros de E/S y cambiar la MMU son privilegiadas.
- La defensa del SO es **preventiva**: el hardware verifica el bit de modo *antes* de ejecutar.

Pregunta de control rápida, respondeme en una línea: **¿quién detecta que la instrucción es privilegiada — el sistema operativo o el hardware?**

Ahora el tema 3, que es el más tomado de toda la unidad. Te armo una visualización para este.

El hardware. Exacto. El SO se entera después, cuando el hardware le entrega el control por el trap. Esa es la idea de fondo: **la protección la impone el hardware, el sistema operativo la configura.** Un SO sin hardware que lo respalde no puede proteger nada.

Tema 3 — Las tres formas de hacer E/S

Te abrí un simulador en el navegador. Si lo cerrás:
es_programada_interrupciones_dma.html — también está linkeado desde visuales/index.html

La intuición

Pusiste la pava. Son 4 minutos. Tenés tres opciones:

1. **Quedarte mirando la pava** hasta que hierva → no hacés nada más.
2. **Que la pava silbe** cuando hierve → mientras tanto hacés otra cosa, pero si silbara cada 5 segundos para avisar cómo viene, no servirías de nada.
3. **Pava eléctrica que se apaga sola** y te avisa una vez al final → te olvidás por completo.

Esas son, en orden, la E/S programada, la E/S por interrupciones y el DMA.

El concepto formal

1. E/S programada (espera ocupada / polling)

El programa emite la llamada al sistema, el kernel llama al driver, el driver inicia la operación y **entra en un ciclo cerrado sondeando continuamente al dispositivo** para ver si terminó.

```
CPU:  [sondea][sondea][sondea][sondea][sondea][sondea]  ← 100% quemada
DISP: [═══════════ transfiriendo ═══════════════════]
```

Defecto: *"mantiene ocupada la CPU sondeando al dispositivo hasta que éste termina"*.

2. E/S controlada por interrupciones

El driver inicia el dispositivo, **le pide que genere una interrupción cuando termine**, y retorna. El SO bloquea al proceso llamador si hace falta y **busca otro trabajo por hacer**.

```
CPU:  [otros procesos][i][otros procesos][i][otros...][i]
                      ↑                  ↑            ↑
DISP: [═══byte═══][═══byte═══][═══byte═══][═══byte═══]
      i = interrupción: la CPU se desvía un instante
```

La CPU se libera, pero paga **una interrupción por cada unidad de datos**.

3. E/S con DMA (Direct Memory Access)

Un chip especial controla el flujo de bits entre la memoria y el controlador **sin intervención constante de la CPU**. La CPU lo configura — cuántos bytes, qué direcciones de dispositivo y memoria, qué operación — y lo deja trabajar. Cuando termina, **genera una única interrupción**.

```
CPU:  [c][═══════ otros procesos ═══════════════][i]
```

```
       ↑                                        ↑
       configura el DMA                   1 sola interrupción
DISP: [═══════════ el DMA transfiere ═══════════]
```

Comparación

	Programada	Interrupciones	DMA
¿CPU libre durante la espera?	**No**	Sí	Sí
Interrupciones	0	**una por byte/bloque**	**1**
Hardware extra	ninguno	controlador de interrupciones	**chip DMA**
Dónde se usa	dispositivos muy simples, arranque	teclado, mouse	**disco, red, placa de video**

Ejemplo resuelto — el que te toman

1000 bytes, el dispositivo entrega **un byte cada 100 µs**, atender una interrupción cuesta **5 µs**.

```
Paso 1  Duración total = 1000 × 100 µs = 100.000 µs = 100 ms
        ⚠ IGUAL EN LOS TRES. La impone el dispositivo.

Paso 2  PROGRAMADA   → CPU = 100 ms           → 100 %
Paso 3  INTERRUPC.   → 1000 × 5 µs = 5 ms     → 5 %
Paso 4  DMA          → 1 × 5 µs = 0,005 ms    → 0,005 %
```

La trampa clásica del examen: preguntan *"¿cuál es más rápido?"*. La respuesta es **ninguno: los tres tardan 100 ms**. Lo que cambia no es la velocidad de la transferencia, sino **cuánta CPU queda libre para otra cosa**.

Metele mano al simulador y probá esto: poné **5 µs por byte** y dejá los **5 µs por interrupción**. Vas a ver saltar el aviso rojo — cuando atender la interrupción cuesta lo mismo que transferir el dato, el método por interrupciones se vuelve inútil. **Ese es el motivo de existir del DMA.**

Referencia

(Tanenbaum, cap. 1.3.5, págs. impresas **27 a 30** → PDF **57 a 60**, y cap. 5.2, págs. impresas **343 a 348** → PDF **373 a 378**).

Preguntas de control

1. El teclado entrega una tecla cada varios cientos de milisegundos, cuando vos escribís. El disco entrega millones de bytes por segundo. **¿Por qué el teclado usa interrupciones y el disco usa DMA?** Pensalo en términos de cuántas interrupciones por segundo generaría cada uno.

2. Verdadero o falso, justificá: *"Usar DMA en lugar de espera ocupada hace que el archivo se lea más rápido."*

Corrección

Las dos correctas. Vamos a ponerle números a la primera, porque el contraste es brutal:

```
TECLADO escribiendo rápido:  ~10 teclas/s
   10 int/s × 5 µs = 50 µs de CPU por segundo  →  0,005 % de la CPU
   Perfecto. Interrupciones.

DISCO a 100 MB/s, con una interrupción por byte:
   100.000.000 int/s × 5 µs = 500 segundos de CPU... por cada segundo
   →  IMPOSIBLE. Harían falta 500 CPUs solo para atender interrupciones.
   Por eso: DMA.
```

Y fijate el matiz que vale la pena: el disco **sí usa interrupciones**, pero **una por transferencia completa**, no una por byte. DMA e interrupciones no son rivales — el DMA *reduce la cantidad* de interrupciones a una sola.

Respuesta 2 impecable. "No más rápido, sino CPU libre para otros procesos". Esa es exactamente la frase que

te da el punto.

Vas 3 de 8. Seguimos.

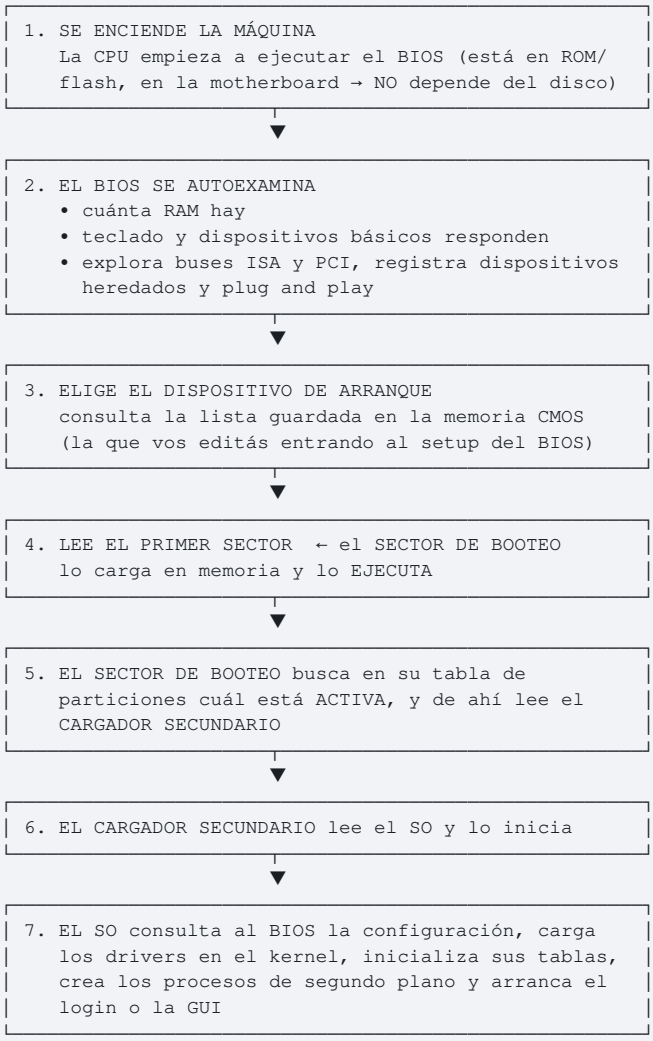
Tema 4 — El arranque y el sector de booteo

La intuición

Problema del huevo y la gallina: el sistema operativo está en el disco, pero para leer el disco hace falta... el sistema operativo. ¿Cómo arranca la máquina si nadie puede leer nada todavía?

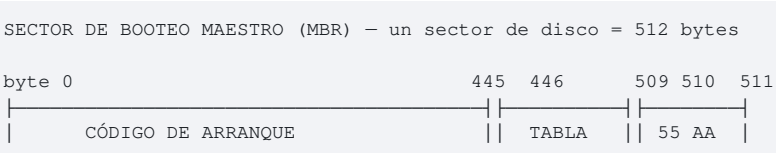
La solución es una **cadena de cargadores**, cada uno más grande y más capaz que el anterior. El primero es tan chico que entra en un chip.

El concepto formal



La pregunta que te van a hacer: *¿por qué hacen falta dos cargadores?* Porque el primero **tiene 446 bytes**. En 446 bytes no entra un cargador que entienda ext4, menús, varios sistemas operativos. Entra solo lo justo para encontrar la partición activa y leer un programa más grande.

Ejemplo resuelto — adentro de los 512 bytes



```
|      446 bytes                          ||    DE    || firma  |
|      (en ensamblador, por el tamaño)    ||PARTICION.|| 2 bytes|
└─────────────────────────────────────────┘└──────────┘└────────┘

                                          4 entradas
                                          × 16 bytes
                                          = 64 bytes

Paso 1   Firma 55AA al final                      =   2 bytes
Paso 2   Tabla: 4 particiones primarias × 16      =  64 bytes
Paso 3   Lo que sobra para el código: 512-64-2    = 446 bytes
Paso 4   Verificación: 446 + 64 + 2               = 512 ✓
```

Cada entrada de 16 bytes guarda: si la partición está **activa**, el **tipo** de sistema de archivos, el **sector de inicio** y la **cantidad de sectores**.

Dato que explica cosas: si la firma `55AA` no está, el BIOS considera que el disco **no es arrancable**. Por eso una sola corrupción de 2 bytes te deja la máquina sin bootear, aunque todos tus archivos estén intactos.

Referencia

(Tanenbaum, cap. 1.3.7, pág. impresa **33** → PDF **63**). El reparto interno de los 512 bytes es complemento mío, no está en el libro — está marcado así en `notebooklm_u01.md`.

Preguntas de control

1. ¿Por qué el BIOS está en ROM o flash y no en el disco duro, como todo lo demás?

2. Un disco tiene 4 particiones primarias y todas tienen un sistema operativo instalado y funcionando. **Ninguna** está marcada como activa. ¿Qué pasa al encender la máquina y en qué paso exacto de la cadena se rompe?

Dale, lo vemos de nuevo desde cero y con otra analogía. El problema es que mezclé dos cosas distintas. Vamos de a una.

Primero: ¿qué es una partición? (olvidate del booteo por un rato)

Imaginate que comprás un **galpón enorme y vacío**.

Podés usarlo así, todo junto. O podés **levantar paredes adentro** y hacer, digamos, 4 depósitos independientes:

```
        GALPÓN (= el disco físico, una sola pieza de hardware)
┌──────────────┬──────────────────┬──────────────┬──────────────┐
│  Depósito 1  │   Depósito 2     │  Depósito 3  │ Depósito 4   │
│              │                  │              │              │
│  Windows     │     Linux        │   tus fotos  │   vacío      │
│  (NTFS)      │     (ext4)       │   (FAT32)    │              │
└──────────────┴──────────────────┴──────────────┴──────────────┘
   ↑ cada uno con su propia organización interna, sin verse entre sí
```

Eso es una partición: un pedazo del disco separado del resto por una "pared". Para el sistema operativo, cada partición se comporta como si fuera un disco aparte.

Lo importante: *las paredes no existen físicamente*. El disco sigue siendo una tira continua de sectores numerados. Las paredes son **puramente información anotada en algún lado**: "del sector 2048 al 500000 es el depósito 1, del 500001 al 900000 es el depósito 2…".

¿Y dónde se anota eso? En la **primera hoja del galpón**. El sector 0. El MBR.

Ahora sí: la historia del arranque, de corrido

Te la cuento como una sola escena, sin cortes.

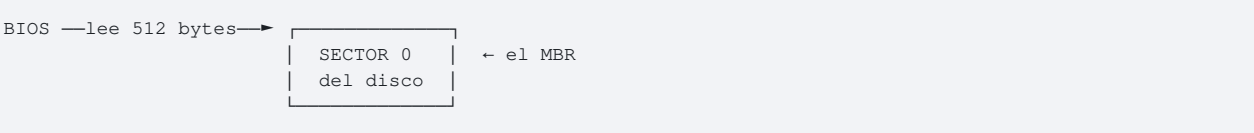
Apretás el botón de encendido.

La CPU despierta y no sabe absolutamente nada. No sabe que existe un disco, ni qué es un archivo. Lo único que sabe hacer es empezar a ejecutar instrucciones desde una dirección fija, que está cableada de fábrica.

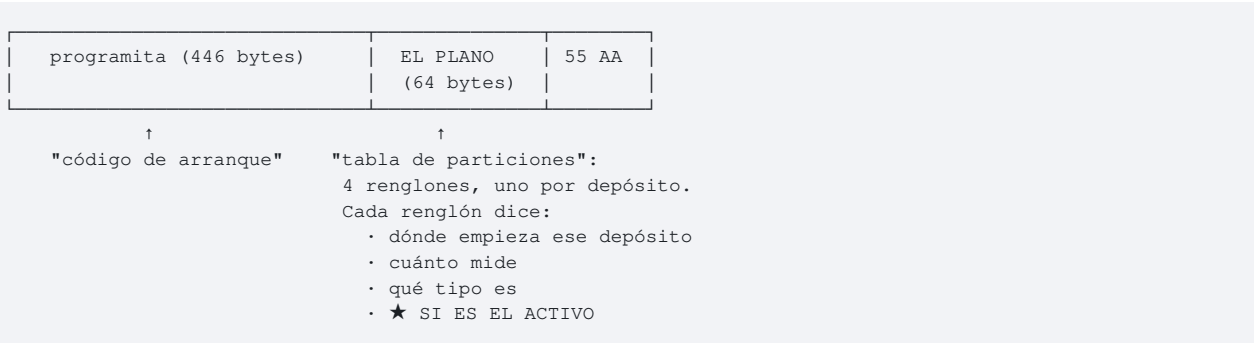
En esa dirección hay un chip soldado a la placa: **el BIOS**. Está ahí, en la placa, no en el disco — justamente porque todavía nadie sabe leer el disco.

El BIOS se despereza: cuenta la RAM, pregunta si el teclado está, revisa qué hay conectado. Después mira una notita que tiene guardada en la memoria CMOS (esa que vos editás cuando entrás al setup): **"el disco de arranque es este"**.

Y acá viene lo único que el BIOS sabe hacer con un disco: **leer el primerísimo sector, 512 bytes, cargarlo en memoria y ejecutarlo a ciegas**. Nada más. No sabe qué es un archivo, no sabe qué es NTFS ni ext4. Lee 512 bytes y los ejecuta.



Esos 512 bytes son el MBR, y adentro tiene dos cosas: un programita chiquito y el plano del galpón.



El programita de 446 bytes se ejecuta y hace **una sola cosa**: mira el plano que tiene al lado, busca el renglón que tiene la estrellita de **activo**, y va a ese depósito.

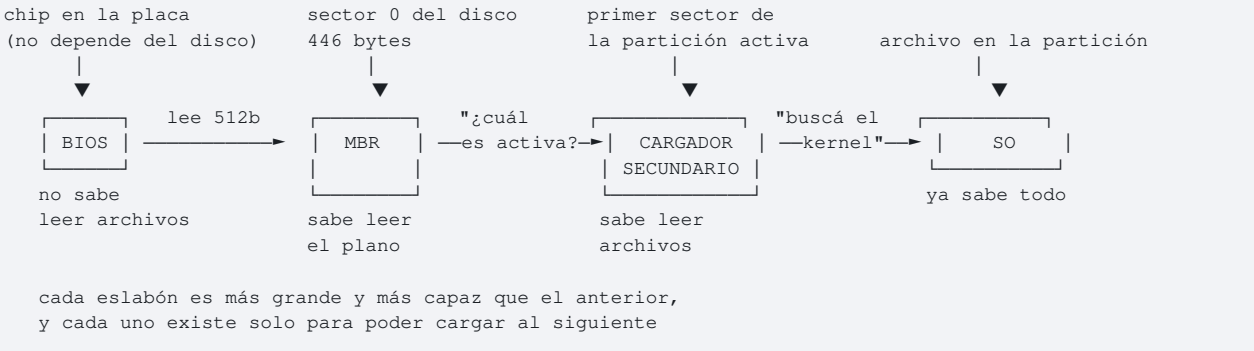
> **¿Por qué tan poquito?** Porque le quedan 446 bytes. Eso es menos que un mensaje de WhatsApp largo. No le entra nada más.

Llega al depósito marcado como activo — digamos el 2, donde está Linux. En el **primer sector de ese depósito** hay otro programa, **el cargador secundario** (GRUB, en Linux). Este ya no está encerrado en 446 bytes: puede ocupar lo que quiera.

Y como es grande, **sí** sabe entender ext4, sí sabe buscar archivos, sí puede mostrarte un menú. Busca el archivo del kernel en la partición, lo carga en memoria, y le pasa el control.

Recién ahí existe el sistema operativo. Carga los drivers, arma sus tablas, levanta los procesos de fondo y te muestra la pantalla de login.

La cadena, en una línea



La idea de fondo, en una frase: cada cargador es demasiado chico para hacer el trabajo final, así que su único trabajo es **buscar a alguien más grande que él**.

Te armo la animación de esto, que creo que es lo que te falta ver.

Las dos, cortito.

---

1. Cómo el BIOS lee el sector 0 sin saber leer archivos

Son dos niveles muy distintos:

```
"Dame el SECTOR 0"            "Dame el archivo /boot/vmlinuz"
        |                                 |
        ▼                                 ▼
Es una orden al hardware.      Es una abstracción construida con
La controladora del disco      software, arriba de miles de sectores.
la entiende sola.              Hay que saber de directorios, tablas,
                               i-nodos o FAT, bloques encadenados...
UNA operación.                 CIENTOS de operaciones + conocer el
                               formato exacto de ESE sistema de archivos.
```

El disco, físicamente, es **una tira de sectores numerados: 0, 1, 2, 3…**. Pedir uno por número es gratis — es lo que el hardware hace nativamente.

El concepto de "archivo" **no existe en el disco**. Lo inventa el sistema operativo. Son esas abstracciones del tema 1: el archivo es una ficción cómoda construida sobre sectores.

> Analogía: el BIOS sabe **abrir el libro en la página 237**. No sabe **buscar el capítulo "Interbloqueos"** — para eso haría falta entender el índice, y cada editorial arma el índice distinto (ext4, NTFS, FAT32…).

Por eso el BIOS puede arrancar cualquier disco del mundo sin saber qué sistema de archivos tiene: nunca lee un archivo.

---

2. Los 446 y los 64 — acá está el nudo, y es bueno que lo preguntes

Ojo, estás comparando dos cosas que no se comparan. Mirá:

```
┌──────────────────────────────────┬──────────────┐
│   446 bytes                      │   64 bytes   │
│   ES UN PROGRAMA                 │   SON DATOS  │
│   instrucciones que SE           │   números    │
│   EJECUTAN                       │   anotados   │
└──────────────────────────────────┴──────────────┘
```

Los 64 bytes no son las particiones. Son la DESCRIPCIÓN de las particiones.

Volvamos al galpón:

```
EL GALPÓN                          EL PLANO
2.000 m²                           una hoja A4

┌──────┬──────┬──────┬──────┐      ┌──────────────────────┐
│ Dep1 │ Dep2 │ Dep3 │ Dep4 │      │ Dep1: del m2 1 al 500│
│      │      │      │      │      │ Dep2: del 501 al 900 │
│ 500m²│ 400m²│ 600m²│ 500m²│      │ Dep3: del 901 al 1500│
└──────┴──────┴──────┴──────┘      │ Dep4: del 1501 al    │
                                   │       2000           │
                                   └──────────────────────┘
```

La hoja **no tiene que medir 2.000 m²** para describir un galpón de 2.000 m². Alcanza con anotar dónde empieza y dónde termina cada cosa.

Lo mismo en el disco. Un renglón de **16 bytes** describe una partición de **200 GB**:

```
UN RENGLÓN = 16 bytes:
   1 byte  → ¿es la activa? (sí/no)
   1 byte  → tipo (NTFS, ext4, FAT32...)
   4 bytes → sector donde EMPIEZA     ← ej: 409.602.048
   4 bytes → cuántos sectores MIDE    ← ej: 300.000.000
   6 bytes → otros datos (geometría)
```

```
  ________
  16 bytes  describen 300.000.000 de sectores = ~150 GB
```

Un número grande ocupa poco. El número 300.000.000 entra en 4 bytes. Para escribir "150 GB" no hacen falta 150 GB.

Y ahora sí, por qué 446 es poco:

```
446 bytes de CÓDIGO  ≈  unas 150 instrucciones de máquina

Con eso hacés:  ✓ leer la tabla de al lado
                ✓ buscar cuál tiene el bit de activa
                ✓ pedirle al BIOS el primer sector de esa partición
                ✓ saltar ahí

Con eso NO hacés: ✗ entender ext4
                  ✗ dibujar un menú de arranque
                  ✗ buscar un archivo entre carpetas
                  ✗ descomprimir el kernel
```

GRUB, el cargador secundario, ocupa **cientos de kilobytes**. Es unas mil veces más grande que los 446 bytes. *Por eso hacen falta dos.*

En una línea: los 446 bytes son chicos porque son **instrucciones y tienen que hacer un trabajo**; los 64 bytes alcanzan de sobra porque son **cuatro renglones de números**.

¿Se destrabó? Si sí, decime **"seguí"** y vamos al tema 5 (llamadas al sistema). Si querés, antes te hago una pregunta de control de esto.

---

1. Un disco tiene una partición de **2 TB**. ¿Cuántos bytes de la tabla de particiones ocupa la descripción de esa partición? ¿Y si fuera de 50 MB?

---

2. Te dicen: *"el MBR no puede arrancar un disco con ext4 porque sus 446 bytes de código no saben leer ext4"*.

¿Verdadero o falso? Justificá.

---

3. Ponele que alguien escribe basura encima de los **primeros 446 bytes** del disco, pero deja intactos los 64 bytes de la tabla y la firma.

- ¿Se perdieron tus archivos?
- ¿Arranca la máquina?
- ¿Qué habría que hacer para repararlo?

Corrección

2 — Correcta. Falso, y por el motivo exacto: el MBR **no lee ext4**, solo salta al primer sector de la partición. El que entiende ext4 es el cargador secundario. Buen punto.

3 — Bien las dos primeras. Archivos intactos ✓, máquina no arranca ✓. El término para la reparación es **reinstalar el código de arranque del MBR** (`bootrec /fixmbr` en Windows, `grub-install` en Linux). Como la tabla de particiones quedó sana, se repara sin tocar un solo archivo tuyo.

---

1 — Acá seguís con el nudo, y es importante que lo destrabes, así que insisto.

Me contestaste **2.000.000.000.000**, que es el tamaño **de la partición**. Pero yo te pregunté cuánto ocupa **la descripción** en la tabla.

Pensalo así:

> Te pido que anotes en un papel: *"el depósito mide 2 billones de metros cuadrados"*.
> **¿Cuánto papel gastaste?** Un renglón.
> No hacen falta 2 billones de hojas para escribir el número "2 billones".

La respuesta es: **16 bytes**. Y si la partición fuera de 50 MB, también **16 bytes**. Y si fuera de 500 TB, **16 bytes**.

```
Partición de   50 MB  →  16 bytes de tabla
Partición de    2 TB  →  16 bytes de tabla
Partición de  500 TB  →  16 bytes de tabla

SIEMPRE 16. El renglón tiene tamaño FIJO.
```

Porque el renglón no guarda los datos de la partición — guarda **dos números**: dónde empieza y cuántos sectores mide. Y un número ocupa lo mismo sea chico o grande (4 bytes), igual que `7` y `9.999.999` ocupan los dos un casillero en una planilla.

La frase para fijarlo: *la tabla de particiones no contiene las particiones, las describe.* Es un índice, no el contenido.

Dato que lo cierra: por eso MBR tiene un límite famoso de **2 TB por partición**. Como el campo "cuántos sectores mide" tiene solo 4 bytes, el número más grande que entra es $2^{32} - 1$, y multiplicado por 512 bytes por sector da ~2 TB. **El límite no viene del disco, viene del tamaño del casillero.** Para pasarlo se inventó GPT, que usa campos de 8 bytes.

Reformulo para confirmar, una línea:

Un disco tiene 4 particiones: 500 GB, 1 TB, 300 GB y 200 GB. ¿Cuántos bytes ocupa la tabla de particiones completa?

Casi. **Entendiste el concepto** — 16 bytes por partición, sin importar el tamaño. Te faltó multiplicar:

```
Partición 1 (500 GB)  →  16 bytes
Partición 2 (1 TB)    →  16 bytes
Partición 3 (300 GB)  →  16 bytes
Partición 4 (200 GB)  →  16 bytes
                         ─────────
                         64 bytes  ← la tabla completa
```

4 × 16 = 64 bytes. Y ahí cerrás el círculo: por eso el MBR reserva exactamente 64 bytes, y por eso **solo hay 4 particiones primarias**. No es un capricho — es que en 64 bytes entran 4 renglones de 16.

```
512 bytes  =  446 (código)  +  64 (4 renglones × 16)  +  2 (firma 55AA)
```

Ahora sí está cerrado el tema 4. Vas **4 de 8**.

Tema 5 — Ensamblador e interrupciones del 8086

La intuición

Un organismo público con muchas ventanillas.

```
VENTANILLA 10h → Video        ¿a qué ventanilla voy?        → INT 10h
VENTANILLA 16h → Teclado
VENTANILLA 17h → Impresora
VENTANILLA 21h → DOS          ¿qué trámite hago ahí?        → AH
                               ¿los datos del trámite?       → los otros registros
```

Vas a la ventanilla (`INT 21h`), decís qué trámite querés (`AH = 09h`, mostrar cadena) y entregás los papeles (`DS:DX` apunta a la cadena). La ventanilla hace el trabajo y te devuelve el resultado en otro registro.

Eso es todo el mecanismo. El resto es saber qué número de trámite es cada cosa.

Por qué aparece el ensamblador justo acá

Porque es el único lenguaje donde vos elegís **qué registro usar**. Y estos servicios se piden *poniendo valores en registros específicos*. En C no tenés forma de decir "poné 09h en AH".

Además, ya lo viste en el tema 4: el código del MBR está en ensamblador **porque tiene 446 bytes**.

Los registros que vas a usar

```
    ┌─────────────────────┐
AX  │   AH   │   AL   │   ← dos mitades de 8 bits, direccionables por separado
    └─────────────────────┘     AH = la de arriba (high), AL = la de abajo (low)
BX  │   BH   │   BL   │
CX  │   CH   │   CL   │
DX  │   DH   │   DL   │
    └─────────────────────┘
       16 bits total

DS, ES  → registros de SEGMENTO (dónde están los datos)
SI, DI, BP, SP → índice y punteros
```

Un par `DS:DX` significa: `DS` dice en qué segmento está el dato, `DX` dice en qué posición dentro de ese segmento. Es una dirección completa partida en dos.

La regla de oro

> **`INT xxh` elige la FAMILIA. `AH` elige la FUNCIÓN dentro de esa familia. Los demás registros llevan los datos.**

Y el apunte de la cátedra avisa explícitamente de la trampa:

> *"**No todo va en AH**: AH selecciona la función; los datos pueden ir en AL, DL, DX, BX, CX, DS:DX, etc."*

INT 21h — funciones de DOS (la más importante)

AH	Qué hace	Dónde van los datos
`01h`	Lee un carácter **con eco**	retorna en `AL`
`02h`	Muestra **un carácter**	el carácter va en **`DL`**
`07h`	Lee **sin eco**	retorna en `AL`
`08h`	Lee sin eco + controla Ctrl-Break	retorna en `AL`
`09h`	Muestra **una cadena**	**`DS:DX`**, la cadena termina en **`$`**
`0Ah`	Lee una cadena a un buffer	`DS:DX` apunta al buffer
`0Bh`	¿Hay entrada disponible?	`AL` = FFh sí, 00h no
`2Ah` / `2Ch`	Fecha / Hora	`CX` año, `DH` mes, `DL` día / `CH:CL:DH` hora
`3Ch`→`42h`	Archivos: crear, abrir, cerrar, leer, escribir, borrar, seek	`BX` handle, `CX` cantidad, `DS:DX` buffer
`4Ch`	**Termina el proceso**	`AL` = código de retorno

El par que más confunden: `02h` → **un** carácter, en **DL**. `09h` → una **cadena**, en **DS:DX**, terminada en **`$`**.

INT 10h — video

AH	Qué hace	Datos	¿Avanza el cursor?
`02h`	Posicionar cursor	`DH`=fila, `DL`=columna, `BH`=página	—

`09h`	Carácter **con atributo** (color)	`AL`=char, `BL`=atributo, `CX`=cantidad	**NO**
`0Ah`	Carácter **sin atributo nuevo**	`AL`=char, `CX`=cantidad	**NO**
`0Eh`	Carácter **y avanza**	`AL`=char	**SÍ**
`13h`	Cadena completa	`ES:BP`=dirección, `CX`=**longitud**, `DH`/`DL`=posición, `BL`=atributo	—

El apunte marca que `09h`, `0Ah`, `0Eh` y `13h` **son las más fáciles de confundir**. La diferencia operativa:

```
09h → pone el carácter con color, pero el cursor NO se mueve
0Eh → pone el carácter y el cursor SÍ avanza  ← la cómoda para escribir seguido
```

INT 16h — teclado (BIOS)

AH	Qué hace
`00h`	Lee tecla, **espera** hasta que haya una. `AL`=ASCII, `AH`=scan code
`01h`	**Consulta** si hay tecla, no bloquea. `ZF=0` hay, `ZF=1` no hay
`02h`	Estado de Shift, Ctrl, Alt, Caps Lock → en `AL`

Fijate el paralelo con el tema 3: `00h` **bloquea** (espera), `01h` **sondea** (polling). Es espera ocupada contra bloqueo, a nivel de teclado.

Ejemplo resuelto — el ejercicio de tu examen

Este es, casi textual, el ejercicio del final práctica que tenés escaneado:

```
ORG 100h
JMP MAIN
MSG  DB  _______________
MAIN:
     MOV DX, MSG
     MOV AH, ____
     INT 21
     RET
```

Resolución paso a paso:

```
Paso 1  ¿Qué quiero hacer?  Mostrar una CADENA (no un carácter suelto).
        → Familia: INT 21h.  Función: AH = 09h.

Paso 2  ¿Qué pide la 09h?   La dirección en DS:DX, y la cadena
        terminada en el signo $.

Paso 3  La cadena:   MSG DB 'ESTA ES LA CADENA A MOSTRAR$'
                                                    ↑
                                          SIN el $ no funciona

Paso 4  MOV DX, MSG   →  ya está: carga el desplazamiento
        MOV AH, 09h   →  selecciona la función
        INT 21        →  DOS recorre desde DS:DX hasta encontrar el $

Paso 5  RET           →  en un .COM con ORG 100h, vuelve a DOS.
                         (En un .EXE sería MOV AX,4C00h / INT 21h)
```

Los 4 errores típicos que marca la cátedra

```
✗ usar DL en lugar de DX        → DL es para UN carácter (02h), no para cadenas
✗ olvidar el $ final            → DOS sigue mostrando basura de memoria
                                  hasta encontrar un $ por casualidad
✗ no cargar DS                  → la dirección apunta a cualquier lado
✗ usar AH=02h para una cadena   → te muestra un solo carácter
```

Para cargar `DS` (en un `.EXE`, donde hace falta):

```
MOV AX, @data     ; no se puede cargar un inmediato directo en DS,
MOV DS, AX        ; por eso se pasa por AX
```

Referencia

(Apunte: `Interrupciones (1).pdf`) y (Apunte: `Tp5 - Tabla Interrupciones.pdf`, Cartilla de Interrupciones UTN-FRM). Este tema **no está en el Tanenbaum** — es material propio de la cátedra.

Te armo el simulador para que practiques armar las llamadas.

Preguntas de control

1. Querés mostrar en pantalla la palabra `FINAL`. Tenés dos caminos: `INT 21h` con `AH=09h`, o `INT 10h` con `AH=13h`.

¿En qué se diferencian en cuanto a cómo le decís al servicio dónde termina la cadena?

---

2. Un compañero escribió esto y no le anda:

```
MSG  DB  'HOLA MUNDO$'
...
     MOV DL, MSG
     MOV AH, 09h
     INT 21h
```

¿Cuál es el error y cómo lo arreglás? (Hay uno solo.)

Dale, lo tiro todo y empiezo de nuevo. Me fui muy rápido con las tablas.

El problema de fondo

En ensamblador **no existe una instrucción para mostrar texto en pantalla**.

Las instrucciones del procesador son nada más que estas: mover un dato, sumar, restar, comparar, saltar. Nada de "escribir". El procesador no sabe qué es una pantalla.

```
MOV   mover un dato de un lado a otro
ADD   sumar
SUB   restar
CMP   comparar
JMP   saltar
...
```

Buscá "mostrar texto" en esa lista. **No está.** No existe.

La solución: rutinas que ya escribió otro

Alguien ya se tomó el trabajo de escribir, instrucción por instrucción, el código que habla con la placa de video. Ese código está **guardado en la memoria de la máquina** desde que arranca.

- Lo escribió el **BIOS** (ese chip del tema 4) → son las rutinas `INT 10h`, `16h`, `17h`
- Y lo escribió el **DOS** → son las rutinas `INT 21h`

Vos no tenés que escribirlas. Solo tenés que **llamarlas**.

¿Y cómo las llamo?

Con la instrucción `INT`. Eso es todo lo que hace `INT`: **ejecuta una rutina que ya está en memoria**.

```
INT 21h   ←  "ejecutá la rutina de DOS"
```

Pero falta algo: ¿cómo le paso los datos?

Acá está el quid.

En C vos escribirías `printf("HOLA")` — le pasás el texto entre paréntesis. **En ensamblador no hay paréntesis ni parámetros.**

Entonces se usa un **acuerdo previo**:

> *"Yo dejo los datos en ciertos registros, y la rutina sabe que tiene que ir a buscarlos ahí."*

Es como dejarle una nota a alguien en un lugar convenido. No se lo decís: lo dejás donde quedaron en que lo ibas a dejar.

Ahora sí, un ejemplo completo. Uno solo.

Quiero que aparezca `HOLA` en la pantalla.

```
MOV DX, MSG      ← línea 1
MOV AH, 09h      ← línea 2
INT 21h          ← línea 3
```

Tres líneas. Vamos una por una, despacio:

Línea 3 — `INT 21h` Es la que hace el trabajo: ejecuta la rutina de DOS. Pero la rutina de DOS sabe hacer **muchas cosas distintas** (mostrar texto, leer el teclado, abrir archivos, dar la hora). Hay que decirle cuál.

Línea 2 — `MOV AH, 09h` Acá le decís cuál. El acuerdo es: **DOS mira el registro `AH` para saber qué querés**.

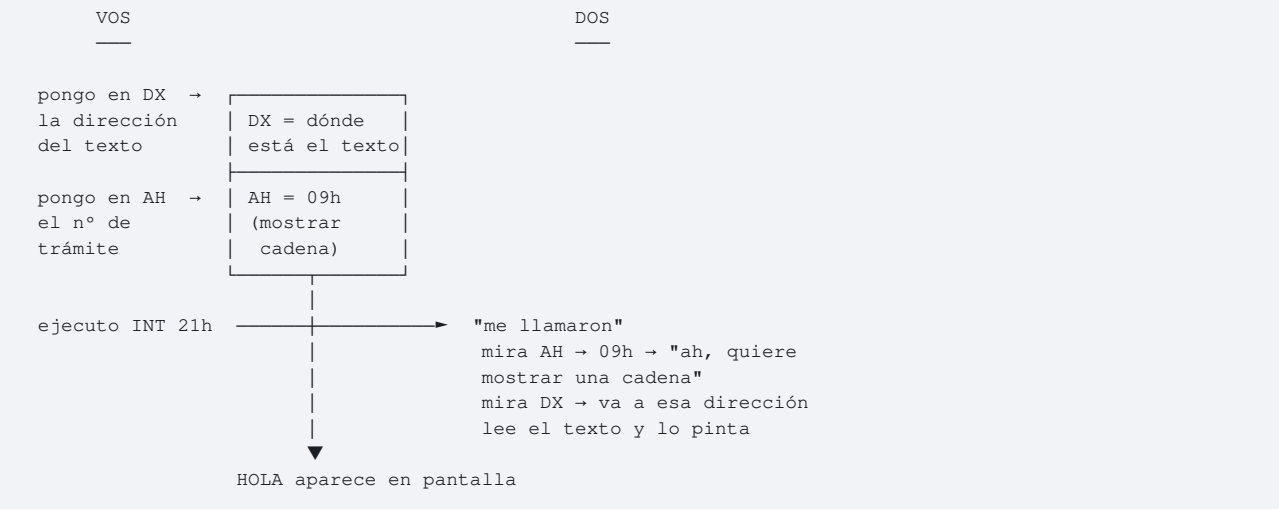
```
si AH vale 09h  →  DOS entiende "mostrar una cadena"
si AH vale 02h  →  DOS entiende "mostrar un carácter"
si AH vale 4Ch  →  DOS entiende "terminar el programa"
```

`AH` es, literalmente, **el número de trámite**.

Línea 1 — `MOV DX, MSG` Ya le dijiste *qué* querés (mostrar una cadena). Ahora falta *cuál* cadena. El acuerdo para la función 09h es: **la cadena tiene que estar apuntada por `DX`**.

Entonces en `DX` ponés la dirección de dónde está tu texto.

En un dibujo



La idea completa, en una frase:

> `INT` llama a una rutina ya escrita. `AH` le dice qué función querés. Los otros registros llevan los datos, en el lugar que esa función tiene acordado.

Todo lo demás —las tablas, los `09h`, `0Eh`, `13h`— es **nada más que la lista de qué número es cada trámite y en qué registro va cada dato**. Es información de consulta, no es concepto. Eso se mira en la cartilla, no se entiende.

Decime en qué punto estás parado:

- **"esto sí lo entendí"** → seguimos y recién ahí miramos qué otras funciones hay
- **"no entiendo qué es un registro"** → volvemos un paso atrás, al tema 2
- **"no entiendo lo de la dirección en DX"** → te explico qué significa que un registro "apunte" a algo
- **"no entiendo para qué sirve todo esto"** → te muestro qué te van a preguntar en el examen y por qué

Perfecto. Ahora un solo paso más.

Segundo ejemplo: la misma mecánica, otro trámite

Ahora quiero mostrar **un solo carácter**: la letra `A`.

```
MOV DL, 'A'      ← el dato
MOV AH, 02h      ← el número de trámite
INT 21h          ← ejecutar
```

Idéntica estructura. **Cambiaron dos cosas:**

```
                 MOSTRAR CADENA        MOSTRAR UN CARÁCTER
                 ──────────────        ───────────────────
nº de trámite    AH = 09h               AH = 02h
dónde va el dato DX                     DL        ← ¡otro registro!
```

Esto es lo único "molesto" del tema: cada función eligió su propio registro para recibir los datos. No hay una regla general. La `09h` quiere la dirección en `DX`; la `02h` quiere el carácter en `DL`.

¿Por qué? Porque las escribieron hace 40 años y cada una eligió lo que le vino bien. **No hay lógica que deducir — se consulta en la cartilla.**

Por qué `DX` en un caso y `DL` en el otro

Esto sí tiene una razón, y es linda:

```
    ┌─────────────────┐
DX  │   DH   |   DL   |     DX entero = 16 bits
    └─────────────────┘     DL solo   =  8 bits
```

- Un **carácter** es 1 byte = 8 bits → entra en `DL`. Sobra con la mitad.
- Una **dirección de memoria** es grande → necesita los 16 bits de `DX` entero.

O sea: el tamaño del registro se eligió según el tamaño del dato. `DL` para una letra, `DX` para una dirección.

Las familias

Hasta acá usamos `INT 21h`, que es la familia de **DOS**. Pero hay otras, escritas por el **BIOS**:

```
INT 10h  →  rutinas de VIDEO       (escritas por el BIOS)
INT 16h  →  rutinas de TECLADO     (escritas por el BIOS)
```

```
INT 17h  →  rutinas de IMPRESORA   (escritas por el BIOS)
INT 21h  →  rutinas de DOS         (escritas por el DOS)
```

Cuidado con esta trampa: el número de `AH` significa cosas distintas según la familia.

```
AH = 09h  con  INT 21h   →   mostrar una CADENA
AH = 09h  con  INT 10h   →   mostrar UN carácter con color
```

Mismo `09h`, dos cosas totalmente distintas. Por eso siempre van de a pares: **primero mirás la `INT`, después el `AH`**.

> Como dos empresas que tienen un "formulario 09". No es el mismo formulario.

Chequeo rápido, 2 preguntas cortas

1. Querés mostrar la letra `Z` en pantalla con DOS. Escribime las tres líneas.

2. Ves este código en un parcial:

```
MOV AH, 4Ch
INT 21h
```

Sabiendo que `4Ch` es "terminar el programa", **¿por qué esta vez no hay una línea que cargue datos en otro registro?**

1 — Correcta. Un detalle de convención nomás: escribilo `02h` con la `h`. Acá da igual porque 02 decimal y 02 hexa son el mismo número, pero con `10` ya no: `10` decimal es `0Ah` hexa. Acostumbrate a poner siempre la `h`.

2 — Bien. No hace falta pasarle datos porque no hay nada que informarle: "cerrá" es toda la orden.

Un matiz que suma punto: `4Ch` **sí acepta un dato opcional**, el *código de retorno*, en `AL`. Es un número que el programa le deja a DOS diciendo "terminé bien" (`00h`) o "terminé con error". Por eso muchas veces vas a ver:

```
MOV AH, 4Ch
MOV AL, 00h      ; "terminé sin errores"
INT 21h
```

que se escribe más corto cargando las dos mitades juntas:

```
MOV AX, 4C00h    ; AH=4Ch y AL=00h de una
INT 21h
```

Esas dos líneas las vas a ver en **todos** los programas. Es el final obligatorio.

Última pieza del tema: el `$`

Volvamos a la cadena. Hay un detalle que te van a tomar sí o sí.

Cuando hacés `MOV DX, MSG`, le estás dando a DOS **dónde empieza** el texto. Pero… **¿dónde termina?**

```
MEMORIA:

dirección → ... H  O  L  A  [?] [?] [?] [?] [?] ...
              ↑
              DX apunta acá

DOS empieza a leer... H, O, L, A, ¿y ahora? ¿sigo? ¿paro?
La memoria no tiene "final". Después de la A hay más bytes,
que son basura de otros programas.
```

La solución de DOS: **vos marcás el final con el signo `$`**.

```
MSG  DB  'HOLA$'
               ↑
          la marca de "hasta acá"
```

DOS lee carácter por carácter y **se detiene cuando encuentra el `$`**. El `$` no se imprime, es solo la señal.

Si te lo olvidás: DOS no sabe dónde parar, sigue leyendo memoria ajena y escupe basura en pantalla hasta toparse con un `$` por pura casualidad. (Eso es lo que probás en el simulador borrando el `$`.)

El ejercicio de tu examen, ahora completo

```
MSG  DB  'ESTA ES LA CADENA A MOSTRAR$'
...
     MOV DX, MSG      ; dónde está el texto
     MOV AH, 09h      ; trámite: mostrar cadena
     INT 21h          ; ejecutar
```

Eso es todo el ejercicio del final. Nada más.

Y ahora sí te repito las dos preguntas de antes, que ahora deberían salir solas:

1. El error en este código de un compañero (hay uno solo):

```
MSG  DB  'HOLA MUNDO$'
     MOV DL, MSG
     MOV AH, 09h
     INT 21h
```

2. ¿Qué pasa si la cadena se declara `MSG DB 'HOLA MUNDO'` (sin el `$`) y todo lo demás está bien?

1 — Correcta. `MOV DX, MSG`. La `09h` quiere la dirección completa de 16 bits.

2 — Acá ajustamos. En mi pregunta "todo lo demás está bien" quería decir que el `DL` ya estaba corregido. El punto era solo el `$`.

Y el efecto no son espacios vacíos — es peor:

```
MSG  DB  'HOLA MUNDO'     ← sin $

MEMORIA:  H O L A _ M U N D O ▓ ▒ ⣿ ? $ ╣ ╗ ⣿ ▒ ▓ ... $ ...
                            └──────────────────────┘    ↑
                            esto también lo imprime      acá recién para,
                                                         por pura casualidad

PANTALLA:  HOLA MUNDO▓▒⣿?$╣╗⣿▒▓
```

DOS **no sabe que tu texto terminó**. Sigue leyendo bytes de la memoria —que son restos de otros programas, código, lo que sea— y los interpreta como caracteres. Sale basura, símbolos raros.

Se detiene recién cuando encuentra un `$` en algún lado de la memoria, por azar. Podrían ser 5 caracteres de basura o 500.

La frase para el examen: *sin el `$`, la función 09h sigue mostrando contenido de memoria hasta encontrar un `$` por casualidad.*

Metele al simulador y borrale el `$` para verlo. Vas **5 de 9**.

Dos preguntas muy buenas. Las dos cortitas.

---

1. No — el código se ejecuta de arriba hacia abajo

Siempre. Línea 1, línea 2, línea 3.

```
MOV DX, MSG      ← se ejecuta PRIMERO
MOV AH, 09h      ← después
INT 21h          ← último
```

Lo que pasó es que **yo te lo expliqué al revés** para que tuviera sentido. Te dije "mirá la línea 3 primero" porque es la que hace el trabajo, y las dos de arriba existen solo para prepararla.

Pensalo como una receta:

```
1. Poné la olla con agua        ← preparación
2. Prendé el fuego              ← preparación
3. Tirá los fideos              ← LA acción
```

Se hace en ese orden. Pero si alguien pregunta "¿qué estás haciendo?", la respuesta es "fideos" — el paso 3. Los pasos 1 y 2 están para que el 3 funcione.

Igual acá: **`INT 21h` es la acción. Las líneas de arriba son la preparación.** Pero se ejecutan en el orden escrito.

---

2. Qué es `DS`

El 8086 parte las direcciones de memoria **en dos pedazos**.

Pensalo como una dirección postal:

```
    DS      :      DX
 ________          ________
 el barrio         el número de casa

 "Palermo"   :   "1234"
```

Ninguno de los dos solo te sirve. "1234" sin el barrio no significa nada, hay un 1234 en cada barrio de la ciudad.

En la máquina:

```
DS  = SEGMENTO DE DATOS    → en qué bloque de memoria están tus datos
DX  = DESPLAZAMIENTO       → a cuántos bytes del inicio de ese bloque
```

Por eso se escribe `DS:DX`, con dos puntos. **Es una sola dirección partida en dos registros.**

```
MEMORIA COMPLETA
┌──────────┬──────────┬──────────┬──────────┐
│ código   │  DATOS   │  pila    │  otros   │
└──────────┴────▲─────┴──────────┴──────────┘
                │
          DS apunta acá (el inicio del bloque de datos)
                │
                └──► DX dice: 30 bytes más adelante está MSG
```

Por qué te lo nombro

Porque la función `09h` busca tu cadena en `DS:DX`. Si `DX` está bien pero `DS` apunta a otro bloque, DOS va a buscar al lugar equivocado y muestra basura. Es uno de los 4 errores típicos.

Cuándo te tenés que preocupar

```
Programa .COM  →  DS ya viene cargado solo. NO hacés nada.
                  (los ejercicios de tu cátedra con ORG 100h son .COM)

Programa .EXE  →  lo tenés que cargar vos:
                  MOV AX, @data
                  MOV DS, AX
```

Y fijate el detalle: **son dos líneas, no una.** No se puede hacer `MOV DS, @data` directo — el procesador no permite cargar un valor fijo en un registro de segmento. Hay que pasar por `AX` como intermediario. Eso también lo preguntan.

Tema 6 — Llamadas al sistema

Arrancamos de lo que ya sabés

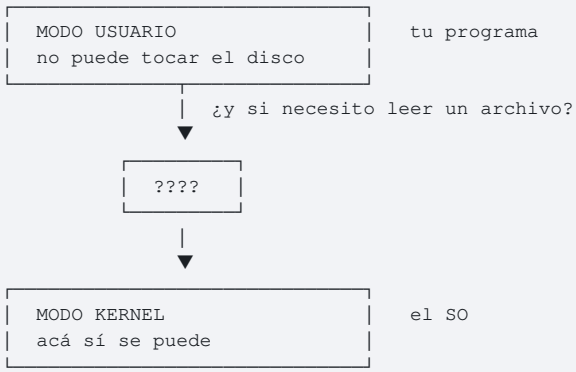
Lo que hiciste en el tema 5 **ya era una llamada al sistema**.

```
MOV AH, 09h
INT 21h        ←  esto es una llamada al sistema de DOS
```

Le pediste un servicio al sistema operativo. Eso es todo el concepto. Lo que cambia hoy es el nombre, el sistema operativo y los detalles — pero el mecanismo es el mismo.

Por qué existen

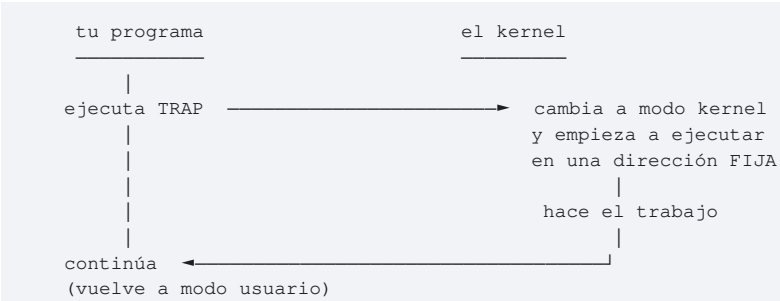
Acordate del tema 2: hay dos modos, usuario y kernel.



Tu programa **necesita** hacer cosas que no tiene permitido hacer. La llamada al sistema es **la puerta**: el único lugar por donde se puede cruzar.

> **Definición para el examen:** las llamadas al sistema son *la interfaz que permite a los programas de usuario solicitar servicios al sistema operativo*.

Cómo se cruza: la instrucción TRAP



`TRAP` se parece a una llamada a procedimiento común, pero **tiene dos diferencias que te van a preguntar**:

	Llamada a procedimiento	TRAP
¿cambia el modo?	NO	SÍ ← pasa a kernel
¿a dónde salta?	a donde quieras	a una dirección FIJA

La segunda es la clave de la seguridad. Si `TRAP` pudiera saltar a cualquier lado, un programa entraría al kernel por donde se le antoje y haría lo que quisiera. Como solo puede saltar a un punto fijo, el kernel controla la entrada.

> Es la misma idea que `INT 21h`: no saltás a cualquier parte del DOS, saltás a su puerta de entrada y ahí el DOS decide.

Las llamadas que te van a pedir (POSIX)

POSIX tiene unas 100. Estas son las que importan:

Para procesos:

`fork`	**Crea un proceso hijo idéntico al padre**
`execve`	**Reemplaza** el programa que corre en el proceso
`exit`	Termina el proceso
`waitpid`	Espera a que un hijo termine

Para archivos:

`open`	**Abre un archivo**
`read`	Lee datos y los pone en un buffer
`write`	Escribe datos desde un buffer
`close`	Cierra el archivo
`lseek`	Mueve el puntero dentro del archivo

Esas cinco de archivos son **exactamente** las que el programa dice que entran en tu examen (*"Análisis de las llamadas al sistema de Linux relacionadas con archivos: open, close, read, write, lseek"*).

`strace` — esto está en tus exámenes reales

En las fotos de exámenes que tenés, la primera pregunta del integrador es literalmente esta:

> *"Se desea utilizar strace para rastrear las llamadas realizadas por `ls -la`, guardando la salida en un archivo llamado salida.txt. La sintaxis del comando es:"*

La respuesta: **`strace -o salida.txt ls -la`**

La opción `-o` manda la salida al archivo. Las otras opciones del múltiple choice (`>`, `|`, `-s`) son los distractores.

Y lo que ves al abrir ese archivo son las llamadas al sistema:

```
openat()      abre archivos o directorios     → abre "." (el directorio actual)
read()        lee desde un descriptor
getdents64()  lee las entradas de un directorio → la lista de archivos
newfstatat()  obtiene info de un archivo      → permisos, tamaño, fechas
write()       escribe en pantalla             → la salida de ls en la terminal
close()       cierra un descriptor
```

Eso es **literalmente ver la frontera** entre tu programa y el kernel. `ls` no sabe leer un disco: le pide todo al sistema operativo, y `strace` te muestra cada pedido.

Ejemplo resuelto — por qué conviene un buffer grande

Querés leer un archivo de **8 KB**.

```
Paso 1   8 KB = 8 × 1024 = 8192 bytes

Paso 2   Con buffer de 512 bytes:
         8192 / 512 = 16 llamadas a read()

Paso 3   Con buffer de 4096 bytes:
         8192 / 4096 = 2 llamadas a read()
```

```
Paso 4   Se leen los MISMOS datos.
         Pero en un caso cruzás la frontera usuario→kernel 16 veces,
         y en el otro 2 veces.
```

Cada cruce cuesta: guardar registros, cambiar de modo, buscar en la tabla del kernel, volver. **Ocho veces menos trabajo de kernel**, leyendo exactamente lo mismo.

> Por eso existen los buffers. No ahorran lectura de disco — **ahorran llamadas al sistema**.

Referencia

(Tanenbaum, cap. 1.6, págs. impresas **49 a 62** → PDF **79 a 92**) y (Apunte: `Comandos_Linux-SO (1).pdf`).

Preguntas de control

1. ¿Cuál es la diferencia entre `TRAP` y una llamada a procedimiento común? (Son dos cosas.)

2. Corrés `strace -o salida.txt ls -la` y en el archivo aparece `write()`. **¿Por qué `ls` necesita pedirle al sistema operativo que escriba en la pantalla, en vez de escribir él directamente?**

Llamadas a procedimiento

Un **procedimiento** es un pedazo de código con nombre que podés ejecutar desde varios lugares. En otros lenguajes se llama función, método o subrutina. Es lo mismo.

Para qué

Si tenés que mostrar un mensaje en 20 lugares distintos del programa, no escribís el código 20 veces. Lo escribís una vez y lo **llamás** 20 veces.

El problema que hay que resolver

```
línea 10:  CALL MOSTRAR     ← saltás al procedimiento
línea 11:  ...                 ¿y cómo volvés acá?

línea 50:  MOSTRAR:
línea 51:    ...código...
línea 52:    RET            ← "volver"... ¿volver A DÓNDE?
```

El procedimiento **no sabe quién lo llamó**. Si lo llamaste desde la línea 10, tiene que volver a la 11. Pero si lo llamaste desde la línea 200, tiene que volver a la 201.

La solución: la pila

Acordate del tema 2, el **apuntador de pila (SP)**. Acá es donde se usa.

```
CALL hace DOS cosas:
  1. guarda en la PILA la dirección de la línea siguiente  ← "la dirección de retorno"
  2. salta al procedimiento

RET hace UNA cosa:
  1. saca de la PILA esa dirección y salta ahí
```

En dibujo:



Por eso se llama pila (stack): lo último que entra es lo primero que sale. Si un procedimiento llama a otro, que llama a otro, las direcciones de retorno se van apilando y se desapilan en orden inverso.

```
PRINCIPAL llama a A
          A llama a B
            B llama a C

PILA:  ┌────────────────┐
       | volver a B     | ← último que entró, primero que sale
       | volver a A     |
       | volver a PRINCIPAL |
       └────────────────┘
```

Y ahora sí, la comparación con TRAP

Esto es lo que te van a preguntar:

```
                          CALL (procedimiento)      TRAP (llamada al sistema)
                          ──────────────────────    ─────────────────────────
¿guarda dir. de retorno?  SÍ, en la pila             SÍ
¿a dónde salta?           a CUALQUIER dirección      a una dirección FIJA
¿cambia el modo?          NO, seguís en usuario      SÍ, pasás a kernel
```

Son casi iguales. Las dos diferencias son todo el asunto de la seguridad.

> Una llamada a procedimiento te mueve **dentro de tu casa**. Un TRAP te lleva **a la oficina del gobierno**, y solo por la puerta principal.

Dato que cierra el círculo con el tema 5

Cuando hacés `INT 21h`, el procesador hace algo **muy parecido a un CALL**: apila la dirección de retorno (y además el PSW, con el bit de modo), salta a la rutina del DOS, y al final la rutina hace un `IRET` ("retorno de interrupción") que desapila todo y te devuelve donde estabas, en modo usuario.

Por eso te decía que `INT` *"llama a una rutina"*: es un CALL con control de seguridad.

Ahora te armo el archivo de continuidad.

Las dos **perfectas**. La 1 completa, con las dos diferencias. La 2 clavada: la pantalla es un dispositivo, y los dispositivos solo se tocan en modo kernel.

Vas **6 de 9**. Los dos que siguen son livianos.

Tema 7 — Historia por generaciones

Cómo estudiar esto

Las fechas no importan. Lo que se toma es **por qué apareció cada invento**. Cada generación resuelve el problema que dejó la anterior:

```
1ª gen → problema: todo a mano, una persona por vez
            ↓
2ª gen → solución: PROCESAMIENTO POR LOTES
         problema nuevo: la CPU se queda quieta esperando E/S
            ↓
3ª gen → solución: MULTIPROGRAMACIÓN + SPOOLING
         problema nuevo: tardás horas en ver el resultado
            ↓
         solución: TIEMPO COMPARTIDO
            ↓
4ª gen → la computadora es personal
```

1ª generación (1945–1955) — tubos al vacío

```
Programación:     cables en tableros de conexiones
Lenguajes:        no existían (ni el ensamblador)
Sistema operativo: NO HABÍA
Modo de trabajo:  te anotabas en una hoja de firmas, entrabas al
                  cuarto, y esperabas horas rezando que no se
                  quemara ninguno de los 20.000 bulbos
```

A principios de los 50 aparecen las **tarjetas perforadas**: ya no cableás, escribís en tarjetas. Lo demás sigue igual.

2ª generación (1955–1965) — transistores y **procesamiento por lotes**

Las máquinas (*mainframes*) se vuelven confiables y carísimas. Aparecen los **operadores profesionales**: el programador ya no toca la máquina.

El problema: se perdía muchísimo tiempo mientras los operadores caminaban llevando tarjetas y papeles de un lado a otro.

La solución — procesamiento por lotes: juntar muchos trabajos, pasarlos todos a una cinta, y que la máquina los ejecute uno atrás del otro sin pausas.

```
(a) tarjetas  →  (b) 1401 las pasa a cinta  →  (c) llevan la cinta
                     (máquina barata)              a mano

(d) 7094 calcula  →  (e) llevan la cinta de vuelta  →  (f) 1401 imprime
    (máquina cara)
```

Aparecen las **tarjetas de control** (`$JOB`, `$FORTRAN`, `$LOAD`, `$RUN`, `$END`). Dato para el examen: **son las precursoras de los shells actuales.**

3ª generación (1965–1980) — circuitos integrados

Acá nacen tres cosas que todavía usás.

IBM System/360: la primera **familia de computadoras compatibles**. Mismo software en la chica y en la grande. Su SO, el OS/360, era tan enorme y lleno de errores que corregir unos introducía otros — *la cantidad total de bugs se mantenía constante*.

🔑 **Multiprogramación** — el concepto más importante de esta generación:

```
PROBLEMA: en datos comerciales, la CPU espera E/S el 80-90% del tiempo

SOLUCIÓN: partir la memoria y meter VARIOS trabajos a la vez

      ┌─────────────┐
      │  Trabajo 3  │   mientras el trabajo 1 espera el disco,
      ├─────────────┤   el 2 usa la CPU
      │  Trabajo 2  │
      ├─────────────┤
      │  Trabajo 1  │
      ├─────────────┤
      │     SO      │
      └─────────────┘
```

Esto **necesita hardware de protección** — si no, un trabajo pisa al otro. Ahí se cierra con el tema 2.

Spooling: leer los trabajos al disco apenas llegan, así cuando una partición queda libre se carga otro al instante.

Tiempo compartido: cada usuario con su terminal. Si hay 20 conectados y 17 están pensando, la CPU atiende a los 3 que piden. Primer sistema: **CTSS** (MIT).

MULTICS: la idea de "computadora como servicio eléctrico" — te enchufás y tenés poder de cómputo. Fracasó comercialmente **pero influyó en todo lo que vino después**.

UNIX: Ken Thompson, que venía de MULTICS, encontró una PDP-7 sin usar y escribió una versión simple para un usuario. Después: System V y BSD (incompatibles) → estándar **POSIX** para unificar → **MINIX** (Tanenbaum, educativo) → **Linux** (Torvalds, inspirado en MINIX).

4ª generación (1980–hoy) — computadoras personales

```
1974  Intel 8080  →  Kildall escribe CP/M
                     (Intel le regaló los derechos: no creía en el disco)

1980  IBM busca SO para la PC → va a Kildall → Kildall no lo atiende
      → vuelve a Bill Gates → Gates compra DOS por 75.000 dólares
      → lo rebautiza MS-DOS

      La jugada: Gates se lo VENDE A LAS FABRICANTES para que
      venga incluido. Kildall intentaba venderlo de a un usuario.
```

La interfaz gráfica: la inventó **Doug Engelbart** (ventanas, iconos, menús, mouse). La adoptó **Xerox PARC**. Steve Jobs la vio ahí y entendió lo que valía — Xerox no. → Lisa (fracaso) → Macintosh (éxito) → Microsoft responde con **Windows**.

El patrón que se toma

> *"La ontogenia recapitula la filogenia"*

Ideas que se inventaron, se descartaron cuando el contexto cambió, **y volvieron una década después en otro contexto**: la programación en ensamblador, la monoprogramación, los directorios de un solo nivel. Desaparecieron de las máquinas grandes y reaparecieron en los sistemas embebidos y las tarjetas inteligentes.

Referencia

(Tanenbaum, cap. 1.2, págs. impresas **7 a 19** → PDF **37 a 49**).

Preguntas de control

1. ¿Cuál es la diferencia entre **procesamiento por lotes** y **multiprogramación**? (Ojo: no son lo mismo, y es una confusión clásica.)

2. ¿Por qué la multiprogramación **no era posible** en la segunda generación, aunque a alguien se le hubiera ocurrido la idea?

Dale. De nuevo, pero contado distinto: **una sola historia**, con una pregunta que la atraviesa.

> **La pregunta de toda la historia:**
> **¿cómo hacemos para que la computadora no esté parada?**

Las computadoras eran carísimas. Cada minuto parada era plata tirada. Toda la historia de los sistemas operativos son intentos de **tapar agujeros de tiempo muerto**.

Vamos agujero por agujero.

Agujero 1: la máquina espera al humano

Años 40 y 50. No hay sistema operativo. Vos reservás un turno, entrás al cuarto, conectás cables en un tablero, y la máquina es tuya por tres horas.

```
┌──────────────────────────────────────────────────┐
| vos cableando | CORRE | vos cableando | CORRE |  |
|   45 minutos  | 3 min |   50 minutos  | 2 min |  |
└──────────────────────────────────────────────────┘
```

```
  ▲                        ▲
la máquina está PARADA mirándote trabajar
```

La máquina trabaja 5 minutos de cada hora. **El resto espera a que el humano haga algo.**

Agujero 2: la máquina espera al operador

Solución al agujero 1: sacar al programador de la sala. Vos entregás tus tarjetas en una ventanilla y te vas a tomar un café. Un **operador profesional** las carga.

Mejoró, pero apareció otro agujero: el operador **camina**. Busca las tarjetas, las carga, saca el papel de la impresora, lo lleva al otro cuarto, vuelve, carga las siguientes…

```
┌──────────────────────────────────────────────────┐
| CORRE | operador caminando | CORRE | caminando|
|       |     4 minutos      |       |          |
└──────────────────────────────────────────────────┘
```

Solución: el procesamiento por lotes.

Junto **un montón** de trabajos, los paso todos a una cinta, y la máquina los ejecuta **uno atrás del otro sin que nadie toque nada**.

```
CINTA con 50 trabajos:

┌──────────────────────────────────────────────┐
| trab1 | trab2 | trab3 | trab4 | trab5 | ...  |
└──────────────────────────────────────────────┘
  ▲
  la máquina pasa del 1 al 2 sola, sin esperar a nadie
```

Lote = montón. Un lote de trabajos es un montón de trabajos juntos.

Esto es procesamiento por lotes: ejecutar trabajos uno después del otro, sin pausas entre ellos.

Fijate bien: **de a uno por vez**. El trabajo 2 empieza cuando el 1 terminó. Nunca hay dos a la vez.

Agujero 3: la CPU espera al disco ☆

Este es el importante.

Ya no hay tiempo muerto *entre* trabajos. Pero hay tiempo muerto **adentro** de cada trabajo.

Un programa de facturación hace esto:

```
leer un registro del disco  →  calcular  →  escribir  →  leer otro  →  ...
      40 milisegundos           0,1 ms       40 ms          40 ms
```

La CPU calcula un instante y después **se queda esperando al disco**. En trabajos comerciales, la CPU espera E/S el **80 o 90 % del tiempo**.

```
UN SOLO TRABAJO EN MEMORIA:

CPU:   ▓▓▓
```

```
┌──────────────┐
│  Trabajo 3   │
├──────────────┤
│  Trabajo 2   │
├──────────────┤
│  Trabajo 1   │
├──────────────┤
│      SO      │
└──────────────┘

CPU:   ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
       1  2  3  1  2  3  1  2  ← siempre hay alguien listo
```

🔑 Lotes vs multiprogramación — la diferencia

Esta es la que te pregunté y la que confunde todo el mundo:

```
PROCESAMIENTO POR LOTES          MULTIPROGRAMACIÓN
──────────────────────────       ─────────────────
UNO en memoria por vez           VARIOS en memoria a la vez

trab1 ──► trab2 ──► trab3        trab1 ┐
                                 trab2 ├─ todos juntos
en fila india                    trab3 ┘

Tapa el agujero ENTRE            Tapa el agujero DENTRO
trabajos                         de un trabajo

(el operador caminando)          (la CPU esperando el disco)
```

No son lo mismo y no se reemplazan: se suman. Un sistema de tercera generación hace lotes *y* multiprogramación al mismo tiempo.

El precio de la multiprogramación

Si hay 3 trabajos en memoria a la vez, **¿qué impide que el trabajo 1 escriba encima del trabajo 2?**

Nada. Salvo que el hardware lo impida.

Por eso la multiprogramación **no era posible antes**: hacía falta **hardware de protección**, y recién lo trajo la tercera generación (la IBM 360). Es lo mismo del tema 2 — modo kernel, modo usuario, el hardware vigilando.

> *Idea que se repite en toda la materia: compartir un recurso siempre exige protección.*

Agujero 4: ahora el que espera sos vos

La máquina está al 100 %. Pero vos entregás tus tarjetas y **el resultado te llega en seis horas**. Si pusiste una coma de más, perdiste el día.

Solución: el tiempo compartido. Cada usuario con su terminal. La CPU va rotando entre todos.

```
20 usuarios conectados, 17 pensando o tomando café
→ la CPU atiende por turnos a los 3 que piden algo

Funciona porque los pedidos interactivos son CORTOS.
```

Es multiprogramación, pero con personas sentadas esperando en vez de trabajos en cinta.

Agujero 5: ya no hay agujero

Años 80. El chip hace que la computadora sea **barata**. Ya no hay que exprimirla: si está parada, no importa, costó 2.000 dólares y no 2 millones.

Ahí el objetivo cambia por primera vez: ya no es *aprovechar la máquina*, es **que sea cómoda para la persona**. Y aparece la interfaz gráfica.

La historia completa en un cuadro

Agujero	¿Quién esperaba?	Solución
1	La máquina esperaba al programador	Sacarlo de la sala: **operadores**
2	La máquina esperaba al operador	**Procesamiento por lotes**
3	La **CPU** esperaba al **disco**	**Multiprogramación** (+ spooling)
4	El **usuario** esperaba el resultado	**Tiempo compartido**
5	Ya no importa: la máquina es barata	**Computadora personal** y GUI

Si te acordás de este cuadro, tenés el tema. Los nombres (IBM 360, MULTICS, UNIX, CP/M, MS-DOS) son anécdotas que cuelgan de acá.

Referencia

(Tanenbaum, cap. 1.2, págs. impresas **7 a 19** → PDF **37 a 49**).

Preguntas de control

1. Un sistema ejecuta trabajos uno atrás del otro desde una cinta, sin pausas, pero con **un solo trabajo en memoria por vez**. ¿Es lotes, es multiprogramación, o es las dos?

2. ¿Por qué la multiprogramación **necesita** hardware de protección? Contestá pensando qué pasaría sin él.

Las dos **correctas**. La 1 es exactamente el punto: lotes sin multiprogramación. La 2 también — y notá que "pisar a otro" es lo mismo que decía el tema 2 cuando hablábamos de multiplexaje en el espacio: **repartir memoria obliga a proteger**.

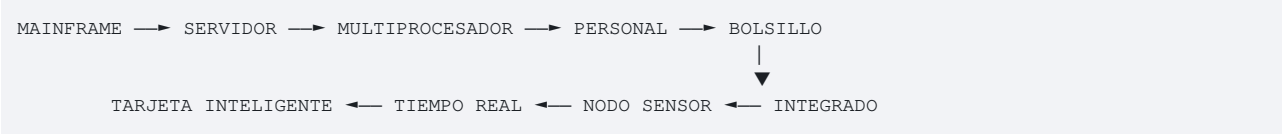
Vas **7 de 9**. Este es cortito.

Tema 8 — Tipos de sistemas operativos

La intuición

Un SO está diseñado para el tamaño de máquina donde va a correr. El de una mainframe con 1000 discos y el de un microondas resuelven problemas **opuestos**.

Tanenbaum describe **nueve**, ordenados de más grande a más chico:



Los nueve, en una línea cada uno

1. De mainframe. Lo que los distingue **no es la CPU, es la capacidad de E/S**: mil discos y millones de gigabytes. Ofrecen tres servicios:

```
por lotes        → trabajos de rutina, sin usuario presente
                   (reclamos de una aseguradora)
transacciones    → MUCHOS pedidos CHICOS: cientos o miles por segundo
```

```
                    (cheques de un banco, reservas de aerolínea)
tiempo compartido→ varios usuarios remotos a la vez
```

2. De servidores. Dan servicio a varios usuarios por red y les permiten compartir recursos. Linux, Windows Server, Solaris, FreeBSD.

3. De multiprocesadores. Varias CPU en un sistema. Son variantes de los de servidor con agregados de comunicación y consistencia. Hoy esto bajó a tu notebook por los chips **multinúcleo**.

4. De computadoras personales. Su trabajo es **dar buen soporte a un solo usuario**. Windows, Linux, macOS.

5. De computadoras de bolsillo. Celulares y PDA — que en la práctica se fusionaron. CPU de 32 bits con modo protegido.

6. Integrados (embebidos, incrustados). Microondas, autos, televisores, reproductores de música.

7. De nodos sensores. Redes de computadoras minúsculas a batería, que se comunican por radio: detectar incendios forestales, vigilar fronteras, medir temperatura. Sistema chico y manejador de eventos. Ejemplo: TinyOS.

8. De tiempo real. Ver abajo — este sí se toma.

9. De tarjetas inteligentes. El más chico: un chip en una tarjeta de crédito. A veces hace **una sola función**, como el pago electrónico.

☆ Lo que te van a preguntar: tiempo real duro vs suave

```
TIEMPO REAL DURO                TIEMPO REAL SUAVE
────────────────                ─────────────────
La acción DEBE ocurrir en       Es aceptable fallar
el instante exacto.             MUY ocasionalmente.
SIN EXCEPCIÓN.

Garantías ABSOLUTAS.            Sin garantías absolutas.

Si falla → desastre físico      Si falla → molestia

Ejemplos:                       Ejemplos:
· robot soldador en una línea   · audio digital
  de montaje                    · multimedia
· aeronáutica                   · teléfonos digitales
· control de procesos
  industriales
· aplicaciones militares
```

El ejemplo del libro, palabra por palabra: *si un robot soldador suelda antes o después de tiempo, el auto se arruina.*

Y el de tiempo real suave: si tu música se corta 20 milisegundos, te molesta. Nadie se muere.

☆ La otra pregunta típica: integrados vs de bolsillo

Parecen lo mismo — los dos son chiquitos. **No lo son**, y la diferencia tiene una consecuencia grande:

```
DE BOLSILLO (celular)           INTEGRADO (microondas)
─────────────────────           ──────────────────────
Instalás apps de terceros       NO se instala nada.
                                Todo el software está en ROM.
         ↓                                ↓
Puede correr software           CERTEZA de que nunca correrá
NO CONFIABLE                    software no confiable
         ↓                                ↓
NECESITA protección             NO necesita protección
                                          ↓
                                el diseño se SIMPLIFICA
```

La frase del libro: *la propiedad principal que diferencia a los sistemas integrados de los de bolsillo es la certeza*

de que nunca se podrá ejecutar software que no sea confiable.

Fijate que es **el mismo razonamiento del tema anterior**, al revés: si no compartís con nadie desconocido, no necesitás protección, y te ahorrás toda esa complejidad.

Referencia

(Tanenbaum, cap. 1.4, págs. impresas **33 a 37** → PDF **63 a 67**).

Preguntas de control

1. El sistema de frenos ABS de un auto y Spotify. **¿Cuál es tiempo real duro y cuál suave? ¿Por qué?**

2. ¿Por qué un sistema operativo **integrado** puede darse el lujo de no implementar protección de memoria, y uno de **celular** no?

Las dos **perfectas**. La 2 clavada: sin software de terceros, no hay de quién protegerse.

Vas **8 de 9**. Último tema.

Tema 9 — Estructuras de los sistemas operativos

La pregunta que define todo

Las cinco estructuras son **cinco respuestas a una sola pregunta**:

> **¿Quién corre en modo kernel?**

Acordate del tema 2: lo que corre en modo kernel puede hacer **cualquier cosa** con la máquina. Entonces:

```
MÁS cosas en modo kernel  →  más rápido, pero un error tumba todo
MENOS cosas en modo kernel →  más seguro, pero más lento
```

Todo el tema es ese intercambio: rendimiento contra confiabilidad. Vamos de un extremo al otro.

---

1. Monolítico — *todo adentro*



Es **un solo programa binario enorme**. Todo junto, enlazado. Cualquier procedimiento puede llamar a cualquier otro directamente.

Por dentro se organiza en tres niveles:

```
programa principal          ← recibe la llamada al sistema
       ↓
procedimientos de servicio  ← uno por cada llamada
       ↓
procedimientos utilitarios  ← ayudan a los anteriores
```

Ventaja: rapidísimo. Llamar a otra parte del SO es una llamada a procedimiento común (acordate del tema 6b: `CALL` y listo).

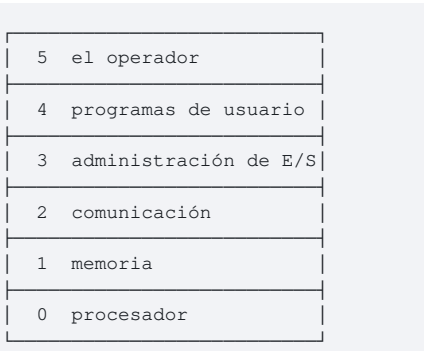
Desventaja: un error en cualquier módulo derriba el sistema entero. El driver de una impresora trucha puede

colgarte la máquina, porque está corriendo con todos los privilegios.

> Es la organización **más común**. Linux es monolítico.

---

2. Por capas — *todo adentro, pero ordenado*



Sigue estando todo en modo kernel, pero **con orden**: la capa 3 no puede meterse con la 1 sin pasar por la 2.

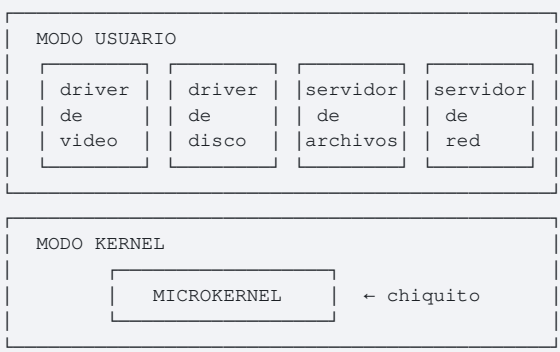
El primer ejemplo fue el **sistema THE**, con 6 capas. **MULTICS** lo generalizó con **anillos concéntricos**: los anillos de adentro tienen más privilegios, y para llamar hacia adentro había que hacer el equivalente a una llamada al sistema.

Ventaja: claridad, se depura ordenado. **Desventaja:** no siempre se puede ordenar todo en capas estrictas, y atravesarlas cuesta.

---

3. Microkernel — *casi todo afuera*

Acá se da vuelta la idea:



Se divide el SO en **módulos chicos y bien definidos**. **Solo el microkernel** corre en modo kernel. Todo lo demás —**incluidos los drivers**— corre como **procesos de usuario comunes**, sin privilegios.

La ganancia es enorme: un driver con errores puede corromper sus propios datos, pero **no puede tumbar el sistema**. Y como es un proceso de usuario, el SO puede detectarlo, matarlo y reiniciarlo al instante.

> Esto importa porque **los drivers son la causa más frecuente de caídas**. Es código escrito por terceros, con menos control de calidad que el kernel.

Ejemplo: **MINIX 3**.

La regla de diseño que acompaña (y que te pueden preguntar):

> **Mecanismo en el kernel, política en el espacio de usuario.**

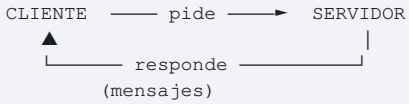
El kernel provee *la capacidad* de hacer algo. *Quién y cuándo* se decide afuera. Ejemplo: el kernel sabe cambiar de proceso (mecanismo); qué proceso sigue lo decide un planificador que vive afuera (política).

El costo: lo que en un monolítico es un `CALL`, acá es **mandar un mensaje a otro proceso**. Mucho más lento.

---

4. Cliente-servidor — *el microkernel, estirado*

Es una variación leve del microkernel. Se distinguen dos tipos de procesos:



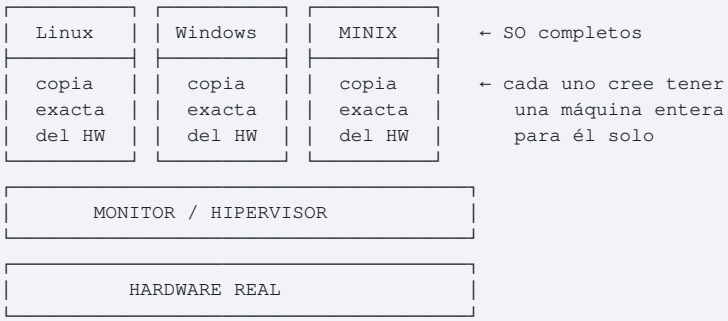
Lo interesante: el cliente **no sabe** si el servidor está en la misma máquina o del otro lado de la red. El mensaje se manda igual.

> Por eso este modelo es **la base conceptual de los sistemas distribuidos** (tu Unidad 8).

---

5. Máquinas virtuales — *varios sistemas operativos a la vez*

Acá se separan dos cosas que en los demás van juntas: la multiprogramación y la máquina extendida.



El monitor **no ofrece una máquina cómoda** — ofrece **copias exactas del hardware real**, con su modo kernel, su modo usuario, sus interrupciones. Arriba de cada copia instalás el SO que quieras.

Dos tipos (esto se toma):

HIPERVISOR TIPO 1	HIPERVISOR TIPO 2
corre sobre el HARDWARE directamente	corre como un PROGRAMA DE USUARIO encima de un SO anfitrión
el clásico, VM/370	VirtualBox sobre Windows

La JVM es otra variante: una arquitectura de computadora **definida por software**, con un intérprete que ejecuta código byte. Por eso un `.class` corre en cualquier máquina que tenga la JVM.

> Esto conecta directo con tu **Unidad 7, Virtualización**.

Comparación final

Estructura	¿Quién en modo kernel?	Ventaja	Desventaja
Monolítica	Todo el SO	Rendimiento	Un error tumba todo
Por capas	Todo, pero ordenado	Claridad	Difícil ordenar, costo por capa

Microkernel	Solo el microkernel	**Confiabilidad**	Lento (mensajes)
Cliente-servidor	Microkernel + mensajes	Se extiende a la red	Latencia
Máquinas virtuales	El hipervisor	Aislamiento total	Sobrecarga

Linux hace trampa, y es la respuesta inteligente: es monolítico, pero con **módulos cargables** — pedazos de código (típicamente drivers) que se insertan y se quitan del kernel **en caliente**, sin reiniciar. Un término medio entre el monolito rígido y el microkernel.

Referencia

(Tanenbaum, cap. 1.7, págs. impresas **62 a 72** → PDF **92 a 102**). También en `fuentes/resumen_catedra_u01.pdf`, sección 1.7.

Te armo la visualización para que veas el intercambio en acción.

Preguntas de control

1. En un microkernel, los drivers corren en **modo usuario**. Pero un driver tiene que hablar con el hardware, y tocar el hardware **requiere modo kernel**.

¿Cómo se resuelve esa contradicción? (Pensalo con lo que ya sabés del tema 6.)

---

2. Te dan a elegir el SO para dos sistemas:

- **(a)** Un servidor de base de datos donde cada microsegundo de latencia cuenta
- **(b)** El control de vuelo de un avión

¿Qué estructura elegís para cada uno y por qué?

Corrección

1 — Correcta. El driver hace una **llamada al sistema** al microkernel, que es quien toca el hardware por él.

Un detalle que lo redondea: el microkernel no le da acceso a *todo* el hardware. Le da acceso **solo a los puertos de su dispositivo**. El driver de video puede tocar la placa de video y nada más.

Eso es exactamente **"mecanismo en el kernel, política afuera"**: el kernel provee la capacidad de tocar un puerto; qué puerto puede tocar cada driver es política, y está controlada.

2a — Acá hay una confusión que conviene desarmar ya, porque es muy fácil de cometer.

Mezclaste dos cosas que se llaman igual:

```
CLIENTE-SERVIDOR como              CLIENTE-SERVIDOR como
ARQUITECTURA DE APLICACIÓN         ESTRUCTURA DEL SO
──────────────────────────         ─────────────────
Tu app le manda consultas SQL      Cómo está organizado el
a un motor de base de datos        kernel POR DENTRO

Es de qué forma se conectan        Es qué corre en modo kernel
dos programas por la red           y qué en modo usuario

NO TIENE NADA QUE VER con          Es el tema de esta unidad
la estructura del SO
```

El servidor de base de datos puede usar la arquitectura cliente-servidor **a nivel aplicación** y correr arriba de un SO **monolítico**. Son capas distintas del problema.

La respuesta correcta para (a) es monolítica. Lo dice tu propia consigna: *"cada microsegundo de latencia cuenta"*. Mirá las barras de la visualización:

```
Monolítica        rendimiento ████████████████████  95
Cliente-servidor  rendimiento ████████              40  ← el más LENTO de todos
```

Cliente-servidor es el peor candidato posible para algo sensible a la latencia, porque **cada operación interna es un mensaje entre procesos** en vez de una llamada a procedimiento.

> Por eso los motores de base de datos serios corren sobre Linux, que es monolítico.

2b — Mismo problema, al revés.

Las capas te dan **claridad para depurar**, sí. Pero volvé a la visualización y apretá el botón rojo en la pestaña "Por capas":

```
Por capas  →  el driver falla  →  EL SISTEMA SE CAE IGUAL
```

En las capas todo sigue corriendo en modo kernel. Ordenado, pero en modo kernel. La protección no mejora un gramo.

La respuesta correcta para (b) es microkernel. Si falla el driver de un sensor en un avión, no podés permitirte que se caiga el sistema de control de vuelo. Necesitás que el driver muera solo y se reinicie sin tocar lo demás.

Y encaja con el dato real: **QNX**, que es un microkernel, es el SO que se usa en aviónica, en sistemas médicos y en autos. No es casualidad.

La regla que te queda

```
¿Qué me duele más perder?

  RENDIMIENTO  →  elegí MONOLÍTICA
  CONFIABILIDAD →  elegí MICROKERNEL
```

Todo lo demás son variantes de esas dos. Y fijate que en tiempo real **duro** (el avión, tema 8) la respuesta es casi siempre microkernel, por la misma razón por la que es duro: no puede fallar.

Terminaste los **9 de 9** temas. ¿Te hago el **cierre de la unidad** — los puntos clave y qué se toma más en los exámenes?

Cierre de la Unidad 1

Los 8 puntos que tenés que tener firmes

1. El SO hace dos cosas que no tienen que ver entre sí. Es **máquina extendida** (crea abstracciones, y la principal es el archivo) y es **administrador de recursos** (hace una *asignación ordenada y controlada*). Multiplexa en el **tiempo** (CPU) y en el **espacio** (memoria, disco).

2. Modo kernel y modo usuario es el cimiento de todo. Un bit del PSW decide. Las instrucciones privilegiadas **las rechaza el hardware antes de ejecutarlas**, no el SO después. Sin este par de modos no hay protección, no hay multiprogramación segura, no hay nada.

3. Los tres métodos de E/S tardan lo mismo. Lo que cambia es la CPU. `Programada 100% · Interrupciones 5% · DMA 0,005%`. La trampa de examen es preguntar cuál es "más rápido": **ninguno**, la duración la impone el dispositivo.

4. La cadena de arranque es una cadena de cargadores, cada uno más grande que el anterior, y cada uno existe solo para cargar al siguiente. `BIOS → MBR (446+64+2) → partición activa → cargador secundario → SO`.

5. `INT` elige la familia, `AH` elige la función, los otros registros llevan los datos. `INT 21h / AH=09h` muestra una cadena desde `DS:DX` terminada en `$`. `AH=02h` muestra **un** carácter desde `DL`.

6. La llamada al sistema es la única puerta al kernel. `TRAP` se diferencia de un `CALL` en dos cosas: **cambia el modo** y **salta a una dirección fija**. Esas dos diferencias son toda la seguridad.

7. La historia es una cadena de agujeros de tiempo muerto que se van tapando. Y la distinción que más confunden: **lotes = uno en memoria por vez, en fila; multiprogramación = varios en memoria a la vez.** La multiprogramación exige hardware de protección.

8. Las cinco estructuras responden a una sola pregunta: ¿quién corre en modo kernel? Y la respuesta es siempre un intercambio: **monolítico = rápido y frágil; microkernel = lento y robusto.**

Lo que más se toma de esta unidad

Sacado de tus 29 fotos de exámenes reales:

🔴 Casi seguro que cae

`strace` — la sintaxis exacta:

```
strace -o salida.txt ls -la
```

Distractores habituales: `>`, `|`, `-s`. Y saber qué llamadas aparecen: `openat`, `read`, `getdents64`, `write`, `newfstatat`, `close`.

Comandos Linux de las cartillas:

Comando	Para qué	
`ls -l /dev | grep '^c'`	identificar dispositivos de **carácter** (con `^b`, de bloque)	
`pgrep firefox` / `ps -ef | grep firefox`	obtener el **PID** de un proceso	
`vmstat`	columna **`id`** = % de tiempo que la CPU está **inactiva**	
`free -h` / `cat /proc/meminfo`	RAM total y libre	
`lsblk -f`	dispositivos de bloque, espacio usado y libre	
`cat /etc/fstab`	**tamaño de bloque** de una partición ext4	

`INT 21h` — completar el código para mostrar una cadena: `AH=09h`, la cadena con `$`, `DS:DX`.

🟠 Aparece seguido

- Las **tres formas de E/S** y cuál conviene según el dispositivo
- **Modo kernel vs usuario** y qué pasa con una instrucción privilegiada
- **Lotes vs multiprogramación** (verdadero/falso con justificación)
- **Tiempo real duro vs suave**, con ejemplos

🟡 Más teórico, suele ser desarrollo

- Las cinco estructuras y su intercambio rendimiento/confiabilidad
- Historia y por qué aparece cada invento

Los errores que ya cometiste y no vas a repetir

Te los anoto porque son exactamente lo que se cobra en un examen:

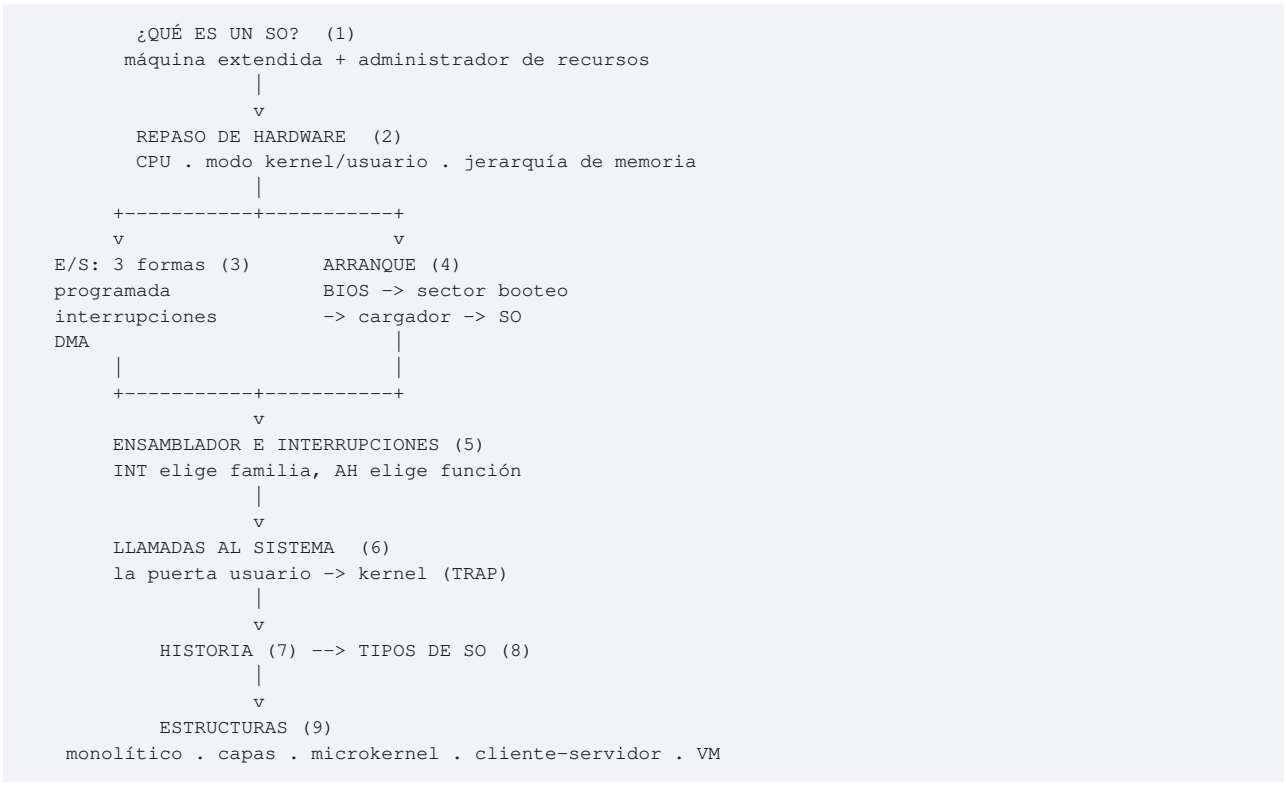
1. ~~"El SO deja de responder"~~ → **el hardware rechaza** la instrucción privilegiada *antes* de ejecutarla
2. ~~Los 64 bytes "son" las particiones~~ → la tabla **describe**, no contiene. 16 bytes por renglón, sea de 50 MB o de 2 TB
3. ~~Cliente-servidor para algo sensible a latencia~~ → es **el más lento**. Y no confundir la estructura del SO con la arquitectura de una aplicación
4. ~~Capas para tolerancia a fallos~~ → en capas **todo sigue en modo kernel**, un driver que falla tumba todo igual

---

Anexo — Resumen condensado de la unidad

La misma unidad en versión corta (la que se armó al cerrar la U1). Además de servir para un repaso rápido, tiene datos del libro que en el chat no se dijeron tal cual: la tabla de llamadas POSIX, POSIX contra Win32, `errno`, la salida de `strace`, el exokernel, Babbage y los comandos de las cartillas.

La idea que atraviesa toda la unidad: el sistema operativo existe para esconder un hardware incómodo y para repartirlo entre varios que lo quieren al mismo tiempo. Los temas 2 a 5 son *el hardware que hay que esconder*. El tema 6 es *la puerta por donde se pide*. Los temas 7 a 9 son *cómo se fue organizando por dentro*.



Resumen · Tema 1 — ¿Qué es un sistema operativo?

Es difícil definirlo más allá de decir que es el software que se ejecuta en modo kernel, y ni siquiera eso es siempre cierto. El problema es que cumple **dos funciones básicas que no están relacionadas entre sí**.

Función 1 — Máquina extendida

La arquitectura de la mayoría de las computadoras a nivel de lenguaje máquina es **primitiva y compleja de**

programar. La **abstracción** es la clave para lidiar con esa complejidad, y la abstracción que todo el mundo entiende es el **archivo**.

```
LEER UN ARCHIVO                 LEER UN ARCHIVO
SIN sistema operativo           CON sistema operativo
--------------------            ----------------------

1. encender motor del disco
2. esperar velocidad
3. mover brazo al cilindro      +----------------------+
4. esperar que pase el sector   | read(fd, buf, nbytes)|
5. leer los bits                +----------------------+
6. verificar código de error
7. apagar el motor
```

"El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los objetos abstractos entonces creados."

Función 2 — Administrador de recursos

Proporciona una **asignación ordenada y controlada** de los procesadores, memorias y dispositivos de E/S entre los programas que compiten por ellos. Lo hace **multiplexando** de dos formas:

	Multiplexaje en el TIEMPO	Multiplexaje en el ESPACIO
Idea	Toman turnos	Cada uno recibe una parte
Recursos	CPU, impresora	Memoria principal, disco
Problema que genera	¿Quién va primero? → planificación (U2)	¿Y si uno pisa al otro? → protección (U3)

Caso del disco: se multiplexa en el **espacio** (cada usuario sus bloques), pero adentro hay **dos recursos distintos**: la superficie magnética se reparte en espacio, y el **brazo**, que es uno solo, se multiplexa en el **tiempo**. De ahí salen los algoritmos de planificación de disco de la Unidad 5.

Ejemplo resuelto: por qué la multiprogramación mejora el uso de la CPU

Un proceso pasa el 80 % de su tiempo esperando E/S. ¿Cuánto se aprovecha la CPU según cuántos procesos haya?

Modelo: si cada proceso espera E/S una fracción `p` del tiempo y hay `n` procesos independientes, la probabilidad de que los n estén esperando a la vez es `p^n`. Entonces el aprovechamiento es **1 – p^n**.

Procesos en memoria	Aprovechamiento de CPU
1	20,00 %
2	36,00 %
3	48,80 %
4	59,04 %
5	67,23 %
10	89,26 %

El rendimiento marginal decrece: cada proceso adicional aporta menos que el anterior y además consume memoria.

Referencia: Tanenbaum, cap. 1.1, págs. impresas 3–7 → PDF 33–37. Y cap. 2.1.7, impresas 93–95.

Resumen · Tema 2 — Repaso del hardware

La CPU y sus registros

Ciclo básico de toda CPU: **obtener** la instrucción de memoria, **decodificarla** para determinar su tipo y operandos, **ejecutarla**, y volver a empezar.

Registro	Qué contiene	Por qué le importa al SO
Contador de programa (PC)	Dirección de la siguiente instrucción a obtener	Para suspender y reanudar un proceso
Apuntador de pila (SP)	Tope de la pila: valores de los procedimientos llamados y no retornados	Cada proceso tiene su propia pila
PSW (palabra de estado)	Bits de código de condición **y el bit de modo**	Acá vive la protección

Modo kernel y modo usuario — el cimiento de todo

```
+-------------------------------------------------+
|  MODO USUARIO                                   |
|  programas comunes                              |
|  . solo un SUBCONJUNTO de instrucciones         |
|  . solo un SUBCONJUNTO del hardware             |
|  . NO puede: apagar interrupciones, tocar E/S,  |
|    acceder a memoria ajena                      |
+--------------------+----------------------------+
                     |
                   TRAP        <- única puerta legal
                     |
                     v
+-------------------------------------------------+
|  MODO KERNEL                                    |
|  el sistema operativo                           |
|  . CUALQUIER instrucción del repertorio         |
|  . TODO el hardware                             |
+-------------------------------------------------+
```

Error típico que hay que evitar: si un programa de usuario ejecuta una instrucción privilegiada (apagar interrupciones, tocar registros de E/S, cambiar la MMU), **NO** es que el sistema operativo se cuelgue y después reaccione. El **hardware** verifica el bit de modo del PSW y **rechaza la instrucción antes de ejecutarla**, generando un trap. Recién ahí el SO toma el control y normalmente mata al proceso.
La protección la impone el hardware; el sistema operativo la configura.

La jerarquía de memoria

```
       ^ más rápido, más caro, menos capacidad
       |
  Registros      ~igual que la CPU      < 1 KB      volátil
  Caché L1/L2    1 a 2 ciclos           KB a MB     volátil
  RAM            decenas de ns          GB          volátil
  Disco          milisegundos           TB          NO volátil
  Cinta          segundos               enorme      NO volátil
       |
       v más lento, más barato, más capacidad
```

Cómo funciona la caché: la memoria principal se divide en **líneas de caché**. Las más usadas se mantienen en una caché rápida dentro o muy cerca de la CPU. Si la línea pedida está, hay **acierto de caché** y no se envía petición por el bus a la memoria principal. Si no, hay **fallo** y se paga el acceso completo. La L1 está dentro de la CPU y se accede sin retraso; la L2 tiene varios MB y cuesta uno o dos ciclos de reloj.

Ejemplo resuelto: tiempo medio de acceso con caché

Caché L1 = 1 ns, RAM = 10 ns, tasa de aciertos = 95 %.

```
Paso 1   95% de las veces:  está en caché       ->  1 ns
Paso 2    5% de las veces:  fallo              ->  1 ns (mirar) + 10 ns (RAM) = 11 ns
Paso 3   promedio ponderado:
         Tm = 0,95 x 1  +  0,05 x 11  =  0,95 + 0,55  =  1,50 ns
Paso 4   sin caché sería 10 ns  ->  10 / 1,50 = 6,67 veces más rápido
```

Variante con mala localidad (50 % de aciertos): Tm = 0,5 × 1 + 0,5 × 11 = **6,00 ns**, mejora de solo **1,67×**. La caché **vale lo que vale la localidad del programa**: el hardware es el mismo, lo que cambió es el software.

El resto

- **Memoria no volátil:** ROM (de fábrica, no se modifica), EEPROM y flash (sí se reescriben), CMOS (volátil, con batería: fecha, hora y configuración del BIOS).
- **Disco:** platos que giran a 5400, 7200 o 10800 rpm. **Pista** = región anular que lee una cabeza. **Cilindro** = todas las pistas para una misma posición del brazo.
- **MMU** (Memory Management Unit): la parte de la CPU que traduce la dirección que genera el programa en la dirección física real. Es el hardware de la memoria virtual (U3).
- **Dispositivos de E/S:** siempre son dos partes, el dispositivo + su **controlador** (chip). El software que le habla al controlador es el **driver**, y corre en modo kernel.
- **Buses:** ISA (viejo) → PCI; USB para dispositivos lentos; SCSI para dispositivos veloces.
- **Plug and play:** permite que el sistema recolecte automáticamente la información de los dispositivos y asigne de forma centralizada los niveles de interrupción y las direcciones de E/S.

Referencia: Tanenbaum, cap. 1.3, págs. impresas 19–33 → PDF 49–63.

Resumen · Tema 3 — Las tres formas de hacer E/S

Lo más importante del tema: los tres métodos transfieren los mismos datos y **tardan exactamente lo mismo**, porque la duración la impone el dispositivo. Lo que cambia radicalmente es **cuánta CPU se gasta**. La trampa de examen es preguntar "¿cuál es más rápido?": **ninguno**.

1. E/S programada (espera ocupada / polling)

El driver inicia la operación y entra en un **ciclo cerrado sondeando continuamente** al dispositivo para ver si terminó. Mantiene la CPU ocupada durante toda la transferencia.

```
CPU:  [sondea][sondea][sondea][sondea][sondea]  <- 100% quemada
DISP: [============ transfiriendo ===========]
```

2. E/S controlada por interrupciones

El driver inicia el dispositivo, le pide que **genere una interrupción cuando termine**, y retorna. El SO bloquea al proceso llamador si hace falta y busca otro trabajo. Paga una interrupción por unidad de datos.

```
CPU:  [otros procesos][i][otros procesos][i][otros...][i]
DISP: [===byte===][===byte===][===byte===][===byte===]
      i = interrupción
```

3. E/S con DMA (Direct Memory Access)

Un chip especial controla el flujo de bits entre la memoria y el controlador **sin intervención constante de la CPU**. La CPU lo configura (cuántos bytes, qué direcciones, qué operación) y lo deja trabajar. Al terminar, genera **una única interrupción**.

```
CPU:  [c][======= otros procesos ===========][i]
       ^                                     ^
       configura el DMA                1 sola interrupción
DISP: [============ el DMA transfiere =========]
```

Canales de datos

En las mainframes la idea se lleva al extremo: procesadores de E/S especializados, con su propio repertorio de instrucciones, que ejecutan programas de canal completos y manejan muchos dispositivos de forma autónoma.

Comparación

	Programada	Interrupciones	DMA
¿CPU libre durante la espera?	No	Sí	Sí
Interrupciones	0	una por byte o bloque	1
Hardware extra	ninguno	controlador de interrupciones	chip DMA
Dónde se usa	dispositivos simples, arranque	teclado, mouse	disco, red, video

Ejemplo resuelto: el que se toma

1000 bytes, el dispositivo entrega un byte cada 100 µs, atender una interrupción cuesta 5 µs.

```
Paso 1  Duración total = 1000 x 100 us = 100.000 us = 100 ms
        IGUAL EN LOS TRES. La impone el dispositivo.

Paso 2  PROGRAMADA   -> CPU = 100 ms           ->  100 %
Paso 3  INTERRUPC.   -> 1000 x 5 us = 5 ms     ->  5 %
Paso 4  DMA          -> 1 x 5 us = 0,005 ms    ->  0,005 %
```

Por qué el teclado usa interrupciones y el disco DMA:
Teclado escribiendo rápido, unas 10 teclas/s → 10 × 5 µs = 50 µs de CPU por segundo = **0,005 %**. Perfecto.
Disco a 100 MB/s con una interrupción por byte → 100.000.000 × 5 µs = **500 segundos de CPU por cada segundo**. Imposible. Por eso DMA.
Ojo: el disco **sí usa interrupciones**, pero una por transferencia completa. El DMA no reemplaza a las interrupciones: **reduce su cantidad a una sola**.

Referencia: Tanenbaum, cap. 1.3.5, impresas 27–30 → PDF 57–60; y cap. 5.2, impresas 343–348 → PDF 373–378.

Resumen · Tema 4 — El arranque y el sector de booteo

El problema del huevo y la gallina: el sistema operativo está en el disco, pero para leer el disco hace falta el sistema operativo. La solución es una **cadena de cargadores**, cada uno más grande y más capaz que el anterior, y cada uno existe solo para poder cargar al siguiente.

Qué es una partición

Un disco es una **tira continua de sectores numerados**. Particionarlo es como levantar paredes dentro de un galpón: las paredes **no existen físicamente**, son **información anotada** en el primer sector.

```
        GALPÓN (= el disco físico)
+---------------+-----------------+--------------+-------------+
|   Depósito 1  |   Depósito 2    |  Depósito 3  | Depósito 4 |
|   Windows     |     Linux       |   tus fotos  |   vacío    |
|   (NTFS)      |     (ext4)      |   (FAT32)    |            |
+---------------+-----------------+--------------+-------------+
   cada uno con su organización interna, sin verse entre sí
```

La cadena de arranque

```
chip en la placa          sector 0 del disco      primer sector de
(no depende del disco)    446 bytes               la partición activa      archivo en la partición
      |                        |                        |                        |
```

```
   v                        v                          v                          v
+------+   lee 512b    +--------+   ¿cuál      +-------------+  "buscá el   +----------+
| BIOS | ------------>  |  MBR   | --es activa?->| CARGADOR    | --kernel"--> |    SO    |
+------+                |        |               | SECUNDARIO |              +----------+
no sabe                 +--------+               +-------------+              ya sabe todo
leer archivos           sabe leer                sabe leer
                        el plano                 archivos

cada eslabón es más grande y más capaz que el anterior,
y cada uno existe solo para poder cargar al siguiente
```

Los siete pasos

1. Se enciende. La CPU ejecuta el **BIOS**, que está en ROM/flash en la motherboard (por eso no depende del disco).
2. El BIOS cuenta la RAM, verifica el teclado, explora los buses ISA y PCI y registra los dispositivos heredados y los plug and play.
3. Determina el dispositivo de arranque consultando la lista guardada en la **memoria CMOS**.
4. Lee el **primer sector**, lo carga en memoria y lo ejecuta. Ese es el **sector de booteo**.
5. El sector de booteo examina su **tabla de particiones**, busca cuál está **activa**, y lee de ahí el **cargador secundario**.
6. El cargador secundario lee el sistema operativo y lo inicia.
7. El SO consulta al BIOS la configuración, carga los drivers en el kernel, inicializa sus tablas, crea los procesos de segundo plano y arranca el login o la GUI.

Ejemplo resuelto: adentro de los 512 bytes

```
SECTOR DE BOOTEO MAESTRO (MBR) — un sector = 512 bytes

byte 0                                  445  446      509 510  511
+-----------------------------------------++----------++--------+
|      CÓDIGO DE ARRANQUE                 ||  TABLA   || 55 AA  |
|      446 bytes                          ||    DE    || firma  |
|      (en ensamblador, por el tamaño)    ||PARTICION.|| 2 bytes|
+-----------------------------------------++----------++--------+
                                            4 entradas
                                            x 16 bytes
                                            = 64 bytes

Paso 1   Firma 55AA al final                      =   2 bytes
Paso 2   Tabla: 4 particiones primarias x 16      =  64 bytes
Paso 3   Lo que sobra para el código: 512-64-2    = 446 bytes
Paso 4   Verificación: 446 + 64 + 2               = 512  OK
```

La confusión más común: los 446 son poco y los 64 alcanzan de sobra. ¿Por qué?
Porque **no se comparan**: los 446 bytes son **un programa** (instrucciones que se ejecutan) y los 64 bytes son **datos** (cuatro renglones de números).

La tabla de particiones no contiene las particiones: las describe. El plano del galpón es una hoja A4, aunque el galpón mida 2000 m². Un renglón de 16 bytes describe una partición de **cualquier** tamaño:

```
UN RENGLÓN = 16 bytes:
   1 byte  -> ¿es la activa? (sí/no)
   1 byte  -> tipo (NTFS, ext4, FAT32...)
   4 bytes -> sector donde EMPIEZA
   4 bytes -> cuántos sectores MIDE
   6 bytes -> otros datos (geometría)
```

Partición de 50 MB → 16 bytes. De 2 TB → 16 bytes. De 500 TB → 16 bytes. **Siempre 16.**
Y una tabla completa de 4 particiones = 4 × 16 = **64 bytes**, sin importar los tamaños.

Dos datos que explican cosas:
• Sin la firma `55AA`, el BIOS considera que el disco **no es arrancable**. Una corrupción de 2 bytes deja la máquina sin bootear aunque todos los archivos estén intactos.
• El **límite de 2 TB por partición** del MBR viene de que el campo "cuántos sectores mide" tiene solo 4 bytes: $2^{32} - 1$ sectores × 512 bytes ≈ 2 TB. **El límite no viene del disco, viene del tamaño del casillero.** Para superarlo se inventó GPT, con campos de 8 bytes.

¿Cómo el BIOS lee el sector 0 si no sabe leer archivos? Porque son dos niveles distintos. "Dame el sector 0" es **una orden al hardware**, que la controladora del disco entiende sola. "Dame el archivo /boot/vmlinuz" es una **abstracción construida por software** sobre miles de sectores, que exige conocer el formato exacto de ese sistema de archivos. El BIOS sabe abrir el libro en la página 237; no sabe buscar un capítulo por el índice. Por eso puede arrancar cualquier disco sin saber qué sistema de archivos tiene: nunca lee un archivo.

Referencia: Tanenbaum, cap. 1.3.7, pág. impresa 33 → PDF 63. El reparto interno de los 512 bytes del MBR es un complemento elaborado: el libro describe el proceso pero no la estructura del sector.

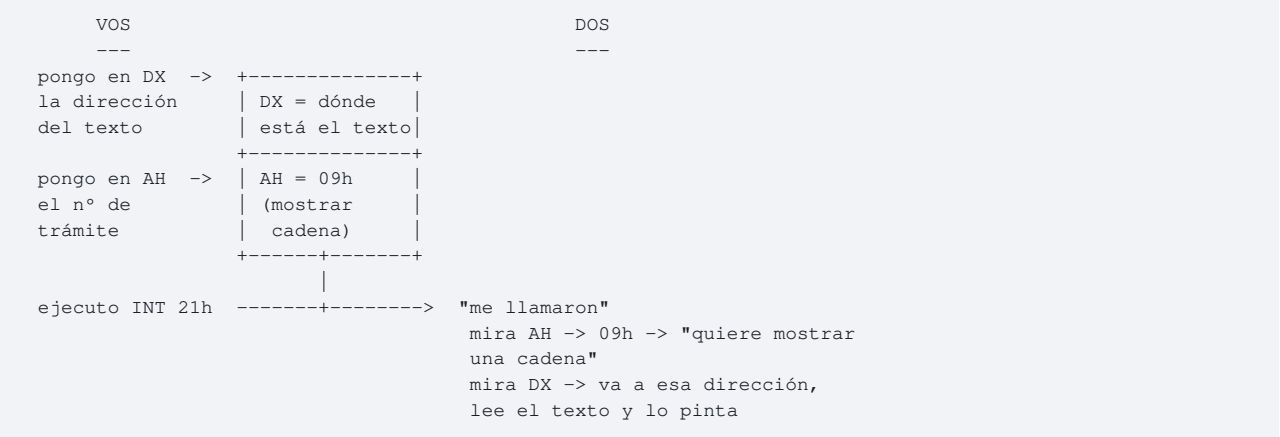
Resumen · Tema 5 — Ensamblador e interrupciones del 8086

El problema de fondo: en ensamblador **no existe una instrucción para mostrar texto en pantalla**. Las instrucciones del procesador son mover, sumar, restar, comparar y saltar. Nada más.
La solución: alguien ya escribió esas rutinas (el BIOS y el DOS) y están en memoria desde que arranca la máquina. Vos solo las llamás.

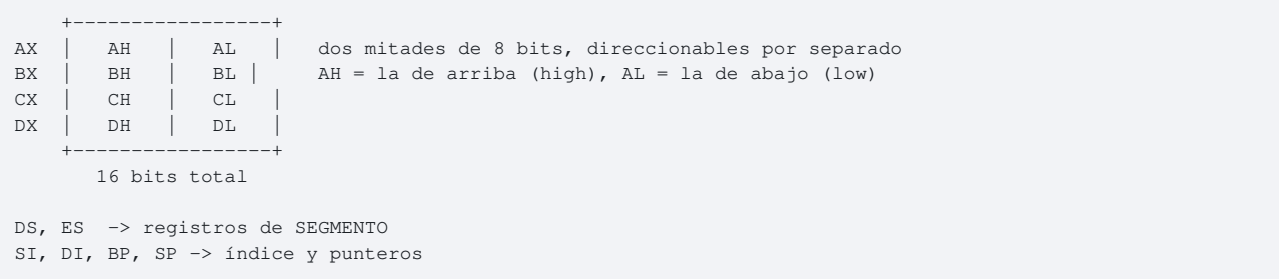
La regla de oro

`INT xxh` elige la FAMILIA. `AH` elige la FUNCIÓN dentro de esa familia. Los demás registros llevan los datos.

El apunte de la cátedra avisa de la trampa: *"No todo va en AH: AH selecciona la función; los datos pueden ir en AL, DL, DX, BX, CX, DS:DX, etc."*



Los registros



Qué significa `DS:DX`: es una sola dirección partida en dos, como una dirección postal. `DS` es el **segmento** (en qué bloque de memoria están los datos) y `DX` el **desplazamiento** (a cuántos bytes del inicio de ese bloque). En un programa `.COM` con `ORG 100h`, `DS` ya viene cargado; en un `.EXE` hay que cargarlo, y como no se admite un valor inmediato en un registro de segmento, se pasa por `AX`: `MOV AX, @data` y después `MOV DS, AX`.

INT 21h — funciones de DOS (la más importante)

AH	Qué hace	Dónde van los datos
01h	Lee un carácter del teclado CON eco	retorna en AL
02h	**Muestra UN carácter**	**el carácter va en DL**
07h	Lee sin eco	retorna en AL
08h	Lee sin eco + controla Ctrl-Break	retorna en AL
09h	**Muestra una CADENA**	**DS:DX, termina en $**
0Ah	Lee una cadena a un buffer	DS:DX (byte 0 capacidad, byte 1 leídos, byte 2+ caracteres)
0Bh	¿Hay entrada disponible?	AL = FFh sí, 00h no
2Ah / 2Ch	Fecha / Hora	CX año, DH mes, DL día / CH:CL:DH
3Ch–42h	Archivos: crear, abrir, cerrar, leer, escribir, borrar, seek	BX handle, CX cantidad, DS:DX buffer
4Ch	**Termina el proceso**	AL = código de retorno (o `MOV AX, 4C00h`)

INT 10h — video

AH	Qué hace	Datos	¿Avanza el cursor?
02h	Posicionar cursor	DH fila, DL columna, BH página	—
09h	Carácter con atributo (color)	AL char, BL atributo, CX cantidad	**NO**
0Ah	Carácter sin atributo nuevo	AL char, CX cantidad	**NO**
0Eh	Carácter y avanza	AL char	**SÍ**
13h	Cadena completa	ES:BP dirección, CX **longitud**, DH/DL posición, BL atributo	—

La cartilla marca que 09h, 0Ah, 0Eh y 13h son **las más fáciles de confundir**.

INT 16h — teclado (BIOS)

- **00h** — lee tecla y **espera** hasta que haya una. Retorna AL = ASCII, AH = scan code.
- **01h** — **consulta** si hay tecla, no bloquea. ZF=0 hay, ZF=1 no hay.
- **02h** — estado de Shift, Ctrl, Alt, Caps Lock, Num Lock, en AL.

Fijate el paralelo con el tema 3: 00h **bloquea**, 01h **sondea** (polling).

INT 17h — impresora

00h imprimir carácter (AL, con DX=0 para LPT1 y DX=1 para LPT2), 01h inicializar, 02h obtener estado (retorna en AH).

Ejemplo resuelto: el ejercicio del final práctica

```
ORG 100h
JMP MAIN
MSG  DB  'ESTA ES LA CADENA A MOSTRAR$'
MAIN:
     MOV DX, MSG      ; dónde está el texto
     MOV AH, 09h      ; trámite: mostrar cadena
     INT 21
     RET
```

Paso 1. Hay que mostrar una CADENA, no un carácter → familia INT 21h, función AH = 09h.
Paso 2. La 09h pide la dirección en DS:DX y la cadena terminada en `$`.
Paso 3. `MOV DX, MSG` carga el desplazamiento.
Paso 4. `MOV AH, 09h` selecciona la función.
Paso 5. `INT 21` ejecuta: DOS recorre desde DS:DX hasta encontrar el `$`.
Paso 6. `RET` vuelve a DOS (en un .EXE sería `MOV AX, 4C00h` + `INT 21h`).

Los 4 errores típicos que marca la cátedra:

```
X usar DL en lugar de DX        -> DL es para UN carácter (02h), no para cadenas
X olvidar el $ final            -> DOS sigue mostrando basura de memoria
                                   hasta encontrar un $ por casualidad
X no cargar DS                  -> la dirección apunta a cualquier lado
X usar AH=02h para una cadena   -> muestra un solo carácter
```

Sobre el segundo: sin el `$`, DOS no sabe que el texto terminó y sigue leyendo bytes de memoria —restos de otros programas— interpretándolos como caracteres. Podrían ser 5 o 500 caracteres de basura.

Dos aclaraciones útiles:
• **El código se ejecuta de arriba hacia abajo**, siempre. Se explica al revés (empezando por el INT) porque esa es la acción, y las líneas de arriba son la preparación. Como una receta: poner la olla, prender el fuego, tirar los fideos.
• `INT` hace algo muy parecido a un `CALL`: apila la dirección de retorno y el PSW, salta a la rutina, y al final la rutina hace `IRET` que desapila todo. Es **un CALL con control de seguridad**.

Referencia: Apunte *Interrupciones (1).pdf* y *Tp5 - Tabla Interrupciones.pdf* (Cartilla de Interrupciones UTN-FRM). Este tema no está en el Tanenbaum: es material propio de la cátedra.

Resumen · Tema 6 — Llamadas al sistema

Definición: las llamadas al sistema son **la interfaz que permite a los programas de usuario solicitar servicios al sistema operativo**. Son la frontera exacta entre el modo usuario y el modo kernel.

Lo que hiciste con `INT 21h` **ya era una llamada al sistema** (de DOS). El mecanismo es el mismo.

La instrucción TRAP

	Llamada a procedimiento (CALL)	TRAP
¿Guarda dirección de retorno?	Sí, en la pila	Sí
¿A dónde salta?	A **cualquier** dirección	A una dirección **FIJA**
¿Cambia el modo?	**No**, seguís en usuario	**Sí**, pasás a kernel

Esas **dos diferencias** son toda la seguridad. Si TRAP pudiera saltar a cualquier lado, un programa entraría al kernel por donde se le antoje. Como solo puede saltar a un punto fijo, el kernel controla la entrada.
Una llamada a procedimiento te mueve *dentro de tu casa*; un TRAP te lleva *a la oficina del gobierno*, y solo

por la puerta principal.

Llamadas a procedimiento: CALL, RET y la pila

Un procedimiento no sabe quién lo llamó. Si lo llamaste desde la línea 10 tiene que volver a la 11; si lo llamaste desde la 200, a la 201. Se resuelve con la **pila**:

```
CALL hace DOS cosas:
  1. guarda en la PILA la dirección de la línea siguiente  <- "dirección de retorno"
  2. salta al procedimiento

RET hace UNA cosa:
  1. saca de la PILA esa dirección y salta ahí

    tu programa                      PILA
    línea 10: CALL MOSTRAR   --+   +----------+
    línea 11: sigue acá  <---+ |   |          |
                             | +-->|    11    |  <- CALL guardó esto
                             |     +----------+
    línea 50: MOSTRAR:       |
    línea 52:   RET  --------+
```

Si un procedimiento llama a otro que llama a otro, las direcciones se **apilan** y se desapilan en orden inverso: lo último que entra es lo primero que sale.

Los once pasos de read()

1. 1 a 3 — El programa mete los parámetros en la pila, **en orden inverso** (nbytes, &bufer, fd). El primero y el tercero por valor; el segundo **por referencia** (se pasa la dirección del buffer, no su contenido).
2. 4 — Llamada al procedimiento de biblioteca `read`, una instrucción de llamada común.
3. 5 — El procedimiento de biblioteca (escrito en ensamblador) coloca el **número de la llamada al sistema** en el registro donde el SO lo espera.
4. 6 — Ejecuta **TRAP**: cambia a modo kernel y empieza en una dirección fija del núcleo.
5. 7 — El kernel usa ese número como **índice en una tabla de apuntadores** a manejadores de llamadas.
6. 8 — Se ejecuta el manejador, que hace la lectura real.
7. 9 — El control vuelve al procedimiento de biblioteca, a la instrucción siguiente al TRAP.
8. 10 — El procedimiento de biblioteca retorna al programa de usuario.
9. 11 — El programa limpia la pila: incrementa el apuntador para eliminar los tres parámetros.

Ojo con el paso 9: la llamada puede **bloquear** al proceso. Si se lee del teclado y no se escribió nada, el proceso se bloquea, el SO ejecuta otro, y recién cuando llega la entrada se completan los pasos 9 a 11.

Valor de retorno: devuelve la cantidad de bytes leídos, que puede ser menor que nbytes si se alcanzó el fin de archivo. Si falla, devuelve **–1** y el número de error queda en la variable global **errno**.

Las llamadas POSIX principales

Procesos	Archivos	Directorios y varias
fork — crea un hijo idéntico al padre **execve** — reemplaza la imagen del proceso **exit** — termina y devuelve el estado **waitpid** — espera a que un hijo termine **kill** — envía una señal	**open** — abre un archivo **read** — lee a un buffer **write** — escribe desde un buffer **close** — cierra **lseek** — mueve el puntero **stat** — información del archivo	**mkdir** / **rmdir** **link** / **unlink** **mount** / **umount** **chdir** — cambia el directorio de trabajo **chmod** — cambia los bits de protección **time** — tiempo desde el 1/1/1970

POSIX frente a Win32

En POSIX la correspondencia entre procedimientos de biblioteca y llamadas al sistema es casi **uno a uno**. La API **Win32** de Windows, en cambio, tiene miles de funciones **deliberadamente desacopladas** de las verdaderas

llamadas al sistema: son procedimientos de biblioteca que pueden invocar o no al kernel. Esa indirección le permite a Microsoft cambiar el kernel sin romper los programas existentes.

strace — aparece en los exámenes

Sintaxis exacta que piden: `strace -o salida.txt ls -la`

La opción `-o` manda la salida al archivo. Distractores habituales: `>`, `|`, `-s`.

Lo que aparece en ese archivo:

openat()	abre archivos o directorios — en este caso "." , el directorio actual
read()	lee contenido desde un descriptor de archivo
getdents64()	lee las entradas de un directorio: la lista de archivos y carpetas
newfstatat() / statx()	obtiene información de archivos: permisos, tamaño, propietario, fechas
write()	escribe la salida de ls en la terminal
close()	cierra un descriptor y libera recursos del kernel

Ejemplo resuelto: por qué conviene un buffer grande

```
Leer un archivo de 8 KB:

Paso 1   8 KB = 8 x 1024 = 8192 bytes
Paso 2   Con buffer de 512 bytes:   8192 / 512  = 16 llamadas a read()
Paso 3   Con buffer de 4096 bytes:  8192 / 4096 =  2 llamadas a read()
Paso 4   Se leen los MISMOS datos, pero se cruza la frontera
         usuario->kernel 16 veces contra 2.
```

Como cada cruce implica los once pasos, el segundo caso hace **ocho veces menos trabajo de kernel**. Los buffers no ahorran lectura de disco: **ahorran llamadas al sistema**.

Referencia: Tanenbaum, cap. 1.6, págs. impresas 49–62 → PDF 79–92. Y apunte *Comandos_Linux-SO (1).pdf*.

Resumen · Tema 7 — Historia por generaciones

Las fechas no importan. Lo que se toma es por qué apareció cada invento. Toda la historia es una cadena de **agujeros de tiempo muerto** que se van tapando, porque las computadoras eran carísimas y cada minuto parada era plata tirada.

Agujero 1 — la máquina espera al humano (1945–1955, tubos al vacío)

```
+-------------------------------------------------+
| vos cableando | CORRE | vos cableando | CORRE |  |
|   45 minutos  | 3 min |   50 minutos  | 2 min |  |
+-------------------------------------------------+
   ^                       ^
   la máquina está PARADA mirándote trabajar
```

- Se programaba conectando cables en **tableros de conexiones**. No existían los lenguajes de programación, **ni siquiera el ensamblador**.
- **No había sistemas operativos.**
- El programador se anotaba en una hoja de firmas, entraba al cuarto y esperaba horas rezando que no se quemara ninguno de los 20.000 bulbos.

- A principios de los 50 aparecen las **tarjetas perforadas**: se escribe en tarjetas en vez de cablear.

Antecedente: **Charles Babbage** (1792–1871) diseñó la primera computadora digital verdadera, la máquina analítica, que nunca funcionó por limitaciones mecánicas. Contrató a **Ada Lovelace** como primera programadora.

Agujero 2 — la máquina espera al operador (1955–1965, transistores)

Se saca al programador de la sala y entran los **operadores profesionales**. Pero el operador **camina**: busca tarjetas, carga, saca papel de la impresora, lo lleva al otro cuarto.

Solución: el PROCESAMIENTO POR LOTES. Juntar un montón (un *lote*) de trabajos, pasarlos a una cinta, y que la máquina los ejecute **uno atrás del otro sin que nadie toque nada**.

```
(a) tarjetas  ->  (b) la 1401 las pasa a cinta  ->  (c) llevan la cinta
                      (máquina barata)                 a mano
(d) la 7094 calcula  ->  (e) llevan la cinta de vuelta  ->  (f) la 1401 imprime
    (máquina cara)
```

Aparecen las **tarjetas de control**: `$JOB` (tiempo máximo, cuenta, programador), `$FORTRAN`, `$LOAD`, `$RUN`, `$END`. **Son las precursoras de los shells e intérpretes de línea de comandos modernos.**

Sistemas operativos típicos: FMS (Fortran Monitor System) e IBSYS.

Agujero 3 — la CPU espera al disco (1965–1980, circuitos integrados)

Ya no hay tiempo muerto **entre** trabajos, pero sí **adentro** de cada uno. En procesamiento de datos comerciales la CPU espera E/S el **80 a 90 %** del tiempo.

```
UN SOLO TRABAJO EN MEMORIA:
CPU:   #.........#.........#.........
       ^ calcula  ^ esperando el disco, sin hacer NADA
```

Solución: la MULTIPROGRAMACIÓN. Partir la memoria y meter varios trabajos a la vez. Cuando uno espera el disco, la CPU se pasa a otro.

```
      MEMORIA
+--------------+
|  Trabajo 3   |
+--------------+        CPU:  ##########################
|  Trabajo 2   |              1  2  3  1  2  3  1  2
+--------------+              siempre hay alguien listo
|  Trabajo 1   |
+--------------+
|      SO      |
+--------------+
```

LOTES vs MULTIPROGRAMACIÓN — la confusión clásica:

```
PROCESAMIENTO POR LOTES          MULTIPROGRAMACIÓN
-------------------------        ------------------
UNO en memoria por vez           VARIOS en memoria a la vez
trab1 -> trab2 -> trab3          todos juntos
en fila india
Tapa el agujero ENTRE            Tapa el agujero DENTRO
trabajos                         de un trabajo
(el operador caminando)          (la CPU esperando el disco)
```

No son lo mismo y no se reemplazan: se suman. Un sistema de tercera generación hace lotes y multiprogramación al mismo tiempo.

Por qué la multiprogramación no era posible antes: con varios trabajos en memoria a la vez, uno puede **pisar la memoria de otro**. Hace falta **hardware de protección**, y recién lo trajo la tercera generación (la IBM 360). Es lo mismo del tema 2: compartir un recurso siempre exige protección.

Otros hitos de la tercera generación:

- **IBM System/360**: la primera **familia de computadoras compatibles** en software. Su SO, el OS/360, era tan enorme que corregir unos errores introducía otros y la cantidad total se mantenía constante.
- **Spooling** (Simultaneous Peripheral Operation On Line): leer los trabajos al disco apenas llegan, para cargar otro apenas una partición queda libre.
- **Tiempo compartido**: cada usuario con su terminal. Si hay 20 conectados y 17 están pensando, la CPU atiende por turnos a los 3 que piden. Funciona porque los comandos interactivos son cortos. Primer sistema: **CTSS** (MIT).
- **MULTICS**: la "utilería para computadora", poder de cómputo como el servicio eléctrico. Éxito parcial comercialmente, pero **influencia enorme**. Primer sistema con memoria virtual y ligado dinámico.
- **UNIX**: Ken Thompson, que venía de MULTICS, escribió una versión simple en una PDP-7. Se fragmentó en System V y BSD → el IEEE creó el estándar **POSIX** para unificar. Después **MINIX** (Tanenbaum, educativo) y **Linux** (Torvalds, inspirado en MINIX).

Agujero 4 — el usuario espera el resultado

La máquina está al 100 %, pero entregás las tarjetas y el resultado llega en seis horas. Una coma de más te cuesta el día. **Solución: el tiempo compartido**, ya mencionado.

Agujero 5 — ya no hay agujero (1980 a hoy, computadoras personales)

Con los circuitos **LSI** la computadora se vuelve **barata**. Por primera vez el objetivo cambia: ya no es aprovechar la máquina, es **que sea cómoda para la persona**.

- **CP/M** (1974): Intel le encarga un SO a Gary Kildall para el 8080 y le cede los derechos porque no cree en las microcomputadoras con disco. Domina el mercado unos cinco años.
- **MS-DOS** (1980): IBM busca un SO para la PC, Kildall no los atiende, vuelven a Bill Gates. Gates compra DOS a Seattle Computer Products por unos 75.000 dólares, lo rebautiza y se lo ofrece a IBM. **La jugada clave:** vendérselo a las fabricantes para que venga incluido con el hardware, frente al intento de Kildall de venderlo de a un usuario.
- **La interfaz gráfica**: la inventó **Doug Engelbart** (ventanas, iconos, menús, ratón), la adoptó **Xerox PARC**, y Steve Jobs entendió su valor al visitarlos. Lisa fracasó por costosa; la **Macintosh** fue un éxito por económica y *amigable*.
- **Windows**: de 1985 a 1995 fue solo un entorno gráfico encima de MS-DOS. Windows 95 fue independiente. Windows NT fue una reescritura de cero en 32 bits, y es la línea de la que descienden 2000, XP y Vista.

El patrón que se toma: *"la ontogenia recapitula la filogenia"*. Ideas que se inventaron, se descartaron cuando el contexto cambió, **y volvieron una década después en otro contexto**: la programación en ensamblador, la monoprogramación, los directorios de un solo nivel. Desaparecieron de las máquinas grandes y reaparecieron en los sistemas embebidos y las tarjetas inteligentes.

La historia completa en un cuadro

Agujero	¿Quién esperaba?	Solución
1	La máquina esperaba al programador	Sacarlo de la sala: **operadores**
2	La máquina esperaba al operador	**Procesamiento por lotes**
3	La **CPU** esperaba al **disco**	**Multiprogramación** + spooling
4	El **usuario** esperaba el resultado	**Tiempo compartido**
5	Ya no importa: la máquina es barata	**Computadora personal** y GUI

Referencia: Tanenbaum, cap. 1.2, págs. impresas 7–19 → PDF 37–49.

Resumen · Tema 8 — Tipos de sistemas operativos

Tanenbaum describe **nueve**, ordenados de la máquina más grande a la más chica.

Tipo	Qué lo caracteriza	Ejemplos
De mainframe	Lo que los distingue **no es la CPU, es la capacidad de E/S**: mil discos y millones de gigabytes. Tres servicios: **por lotes** (sin usuario presente), **procesamiento de transacciones** (muchos pedidos chicos, cientos o miles por segundo) y **tiempo compartido**.	OS/390
De servidores	Dan servicio a varios usuarios por red y permiten compartir recursos de hardware y software.	Solaris, FreeBSD, Linux, Windows Server
De multiprocesadores	Varias CPU en un sistema: computadoras en paralelo, multicomputadoras o multiprocesadores. Variantes de los de servidor con agregados de comunicación y consistencia.	Windows, Linux
De computadoras personales	Su trabajo es **dar buen soporte a un solo usuario**.	Linux, Windows, macOS
De bolsillo	Celulares y PDA, que se fusionaron. CPU de 32 bits con modo protegido.	Symbian, Palm OS
Integrados	Controlan dispositivos que no se consideran computadoras. Todo el software está en ROM.	QNX, VxWorks
De nodos sensores	Computadoras minúsculas a batería que se comunican por radio. SO chico y manejador de eventos.	TinyOS
De tiempo real	El **tiempo** es el parámetro clave. Ver abajo.	—
De tarjetas inteligentes	El más chico: un chip en una tarjeta. A veces una sola función, como el pago electrónico.	—

Tiempo real duro vs suave — esto se toma

```
TIEMPO REAL DURO                TIEMPO REAL SUAVE
----------------                -----------------
La acción DEBE ocurrir en       Es aceptable fallar
el instante exacto.             MUY ocasionalmente.
SIN EXCEPCIÓN.
Garantías ABSOLUTAS.            Sin garantías absolutas.
Si falla -> desastre físico     Si falla -> molestia

Ejemplos:                       Ejemplos:
. robot soldador en una línea   . audio digital
  de montaje                    . multimedia
. aeronáutica                   . teléfonos digitales
. control de procesos
. aplicaciones militares
```

El ejemplo del libro: *si un robot soldador suelda antes o después de tiempo, el auto se arruina.* En cambio, si la música se corta 20 milisegundos, molesta pero no pasa nada.

Integrados vs de bolsillo — la otra pregunta típica

```
DE BOLSILLO (celular)           INTEGRADO (microondas)
---------------------           ----------------------
Instalás apps de terceros       NO se instala nada.
```

```
                                Todo el software está en ROM.
         |                                |
         v                                v
Puede correr software            CERTEZA de que nunca correrá
NO CONFIABLE                     software no confiable
         |                                |
         v                                v
NECESITA protección              NO necesita protección
                                          |
                                          v
                                 el diseño se SIMPLIFICA
```

Frase del libro: *la propiedad principal que diferencia a los sistemas integrados de los dispositivos de bolsillo es la certeza de que nunca se podrá ejecutar software que no sea confiable.*

Referencia: Tanenbaum, cap. 1.4, págs. impresas 33–37 → PDF 63–67.

Resumen · Tema 9 — Estructuras de los sistemas operativos

Las cinco estructuras son **cinco respuestas a una sola pregunta: ¿quién corre en modo kernel?**
Más cosas en modo kernel → más rápido, pero un error tumba todo.
Menos cosas en modo kernel → más seguro, pero más lento.
Todo el tema es ese intercambio: rendimiento contra confiabilidad.

1. Monolítica — todo adentro

Es **un solo programa binario enorme**, todo enlazado, corriendo en modo kernel. Cualquier procedimiento puede llamar a cualquier otro directamente. Por dentro, tres niveles:

```
   programa principal          <- recibe la llamada al sistema
          |
   procedimientos de servicio  <- uno por cada llamada
          |
   procedimientos utilitarios  <- ayudan a los anteriores
```

Ventaja: rapidísimo, llamar a otra parte del SO es una llamada a procedimiento común. **Desventaja:** un error en cualquier módulo **derriba el sistema entero**. Es la organización más común; Linux es monolítico.

2. Por capas — todo adentro, pero ordenado

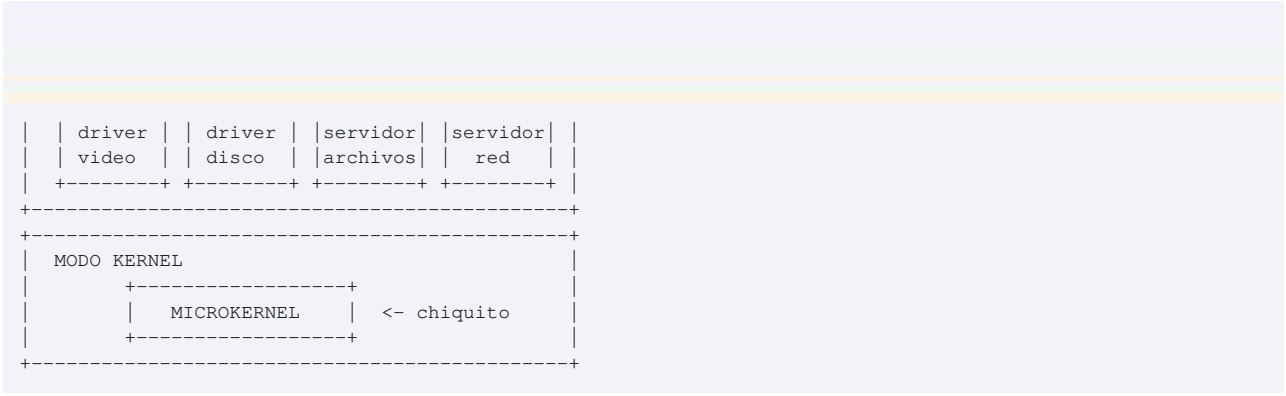
```
+--------------------------+
|  5  el operador          |
|  4  programas de usuario |
|  3  administración de E/S|   <- acá viven los drivers
|  2  comunicación         |
|  1  memoria              |
|  0  procesador           |
+--------------------------+
      cada capa usa SOLO la de abajo
```

El primer ejemplo fue el **sistema THE**, con 6 capas. **MULTICS** lo generalizó con **anillos concéntricos**, donde los de adentro tienen más privilegios y llamar hacia adentro equivale a una llamada al sistema. **Ventaja:** claridad y depuración ordenada. **Desventaja:** no todo se deja ordenar en capas estrictas, y atravesarlas cuesta.

Ojo: las capas **no dan tolerancia a fallas**. Todo sigue corriendo en modo kernel, así que un driver que falla tumba el sistema igual que en la monolítica. Lo que mejora es la claridad para depurar.

3. Microkernel — casi todo afuera

```
+-----------------------------------------------+
|  MODO USUARIO                                 |
|  +--------+ +--------+ +--------+ +--------+ |
```



Solo el microkernel corre en modo kernel. Todo lo demás, **incluidos los drivers**, corre como procesos de usuario ordinarios sin privilegios. Un driver con errores puede corromper sus propios datos pero **no puede tumbar el sistema**: el SO lo detecta, lo mata y lo reinicia. Ejemplo: **MINIX 3**.

Importa porque los drivers son la causa más frecuente de caídas: es código de terceros con menos control de calidad que el kernel.

Regla de diseño: mecanismo en el kernel, política en el espacio de usuario. El kernel provee la capacidad de hacer algo; quién y cuándo se decide afuera.

Costo: lo que en un monolítico es un CALL, acá es **mandar un mensaje entre procesos**. Mucho más lento.

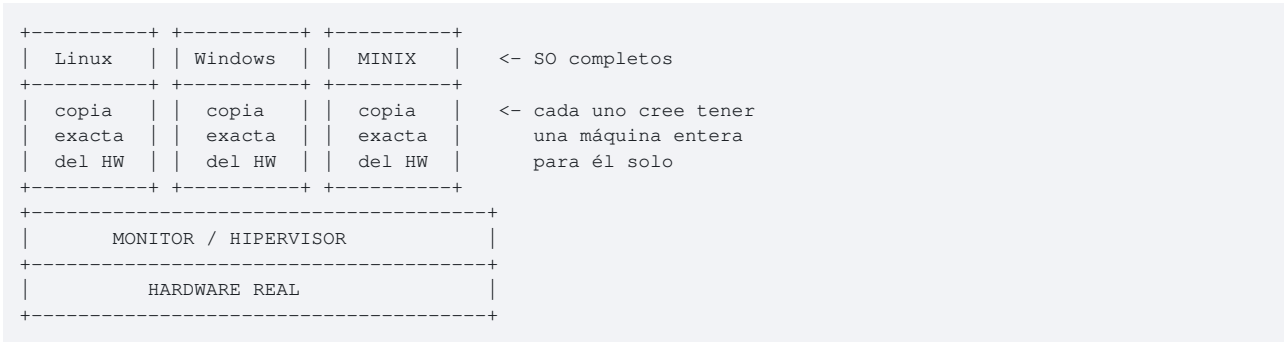
> **¿Cómo toca el hardware un driver que corre en modo usuario?** Haciendo una **llamada al sistema** al microkernel, que lo hace por él. Y el microkernel le da acceso **solo a los puertos de su dispositivo**, no a todo el hardware. Eso es exactamente "mecanismo en el kernel, política afuera".

4. Cliente-servidor — el microkernel, estirado

Variación leve del microkernel: **servidores** que proveen un servicio y **clientes** que lo usan, comunicados por **paso de mensajes**. Lo interesante: el cliente **no sabe** si el servidor está en la misma máquina o del otro lado de la red. Por eso es **la base conceptual de los sistemas distribuidos** (Unidad 8).

> **No confundir:** cliente-servidor como **estructura del SO** (cómo está organizado el kernel por dentro) es otra cosa que cliente-servidor como **arquitectura de aplicación** (tu app mandándole consultas SQL a un motor de base de datos). Son capas distintas del problema: un servidor de base de datos puede usar la arquitectura cliente-servidor a nivel aplicación y correr sobre un SO monolítico.

5. Máquinas virtuales — varios sistemas operativos a la vez



El monitor separa dos funciones que en los demás van juntas: la multiprogramación y la máquina extendida. **No ofrece una máquina cómoda**, ofrece **copias exactas del hardware real**, con su modo kernel, su modo usuario y sus interrupciones. El ejemplo clásico es el **VM/370** de IBM.

Hipervisor TIPO 1	Hipervisor TIPO 2
Corre sobre el **hardware** directamente. El monitor tradicional, VM/370.	Corre como un **programa de usuario** encima de un SO anfitrión, e interpreta el conjunto de instrucciones. VirtualBox sobre Windows.

La **JVM** es otra variante: una arquitectura de computadora definida por software, con un intérprete que ejecuta código byte. Por eso un programa compilado a código byte corre en cualquier máquina que tenga la JVM.

Exokernel: la idea opuesta a la máquina virtual. En vez de dar a cada usuario una copia de la máquina completa, le da **un subconjunto de los recursos reales** (por ejemplo, los cilindros 0 a 1023 del disco). Ahorra la capa de traducción entre direcciones virtuales y reales.

Comparación final

Estructura	¿Quién en modo kernel?	Ventaja	Desventaja	Ejemplo
Monolítica	Todo el SO	Rendimiento	Un error tumba todo	Linux, UNIX
Por capas	Todo, ordenado	Claridad	Difícil ordenar, costo por capa	THE, MULTICS
Microkernel	Solo el microkernel	**Confiabilidad**	Lento (mensajes)	MINIX 3, QNX
Cliente-servidor	Microkernel + mensajes	Se extiende a la red	Latencia	Base de los distribuidos
Máquinas virtuales	El hipervisor	Aislamiento total	Sobrecarga	VM/370, hipervisores

Linux hace trampa, y es la respuesta inteligente: es monolítico, pero con **módulos cargables** —pedazos de código, típicamente drivers, que se insertan y se quitan del kernel **en caliente**, sin reiniciar—. Un término medio entre el monolito rígido y el microkernel.

La regla para elegir: ¿qué me duele más perder? Rendimiento → **monolítica**. Confiabilidad → **microkernel**. En tiempo real duro la respuesta es casi siempre microkernel, por la misma razón por la que es duro: no puede fallar. (QNX, un microkernel, es el SO de aviónica, sistemas médicos y autos.)

Referencia: Tanenbaum, cap. 1.7, págs. impresas 62–72 → PDF 92–102.

Resumen · Cierre — los 8 puntos que hay que tener firmes

1. **El SO hace dos cosas que no tienen que ver entre sí.** Máquina extendida (abstracciones, la principal es el archivo) y administrador de recursos (*asignación ordenada y controlada*). Multiplexa en el **tiempo** (CPU) y en el **espacio** (memoria, disco).
2. **Modo kernel y modo usuario es el cimiento de todo.** Un bit del PSW decide. Las instrucciones privilegiadas **las rechaza el hardware antes de ejecutarlas**, no el SO después.
3. **Los tres métodos de E/S tardan lo mismo. Lo que cambia es la CPU:** 100 % / 5 % / 0,005 %. La trampa es preguntar cuál es "más rápido": ninguno.
4. **La cadena de arranque es una cadena de cargadores**, cada uno más grande que el anterior y existiendo solo para cargar al siguiente: BIOS → MBR (446+64+2) → partición activa → cargador secundario → SO.
5. **INT elige la familia, AH elige la función, los otros registros llevan los datos.** INT 21h / AH=09h muestra una cadena desde DS:DX terminada en `$`; AH=02h muestra un carácter desde DL.
6. **La llamada al sistema es la única puerta al kernel.** TRAP se diferencia de un CALL en dos cosas: cambia el modo y salta a una dirección fija. Esas dos diferencias son toda la seguridad.
7. **La historia es una cadena de agujeros de tiempo muerto que se van tapando.** Y la distinción clave: **lotes = uno en memoria por vez, en fila; multiprogramación = varios en memoria a la vez**. La multiprogramación exige hardware de protección.

8. **Las cinco estructuras responden a: ¿quién corre en modo kernel?** Y la respuesta es siempre un intercambio: monolítico = rápido y frágil; microkernel = lento y robusto.

Resumen · Lo que más se toma de esta unidad

Casi seguro que cae

strace — la sintaxis exacta: `strace -o salida.txt ls -la`. Distractores habituales: `>`, `|`, `-s`.

Comandos Linux de las cartillas:

Comando	Para qué
`ls -l /dev | grep '^c'`	identificar dispositivos de **carácter** (con `^b`, de bloque)
`pgrep firefox` / `ps -ef | grep firefox`	obtener el **PID** de un proceso
`vmstat`	columna **id** = % de tiempo que la CPU está **inactiva**
`free -h` / `cat /proc/meminfo`	RAM total y libre
`lsblk -f`	dispositivos de bloque, espacio usado y libre
`cat /etc/fstab`	**tamaño de bloque** de una partición ext4
`kill -STOP PID` / `kill -CONT PID`	suspender y reanudar un proceso
`iostat`	actividad, velocidad, espera y utilización de E/S

INT 21h — completar el código para mostrar una cadena: AH=09h, la cadena con `$`, DS:DX.

Aparece seguido

- Las tres formas de E/S y cuál conviene según el dispositivo
- Modo kernel vs usuario y qué pasa con una instrucción privilegiada
- Lotes vs multiprogramación (verdadero/falso con justificación)
- Tiempo real duro vs suave, con ejemplos

Más teórico, suele ser desarrollo

- Las cinco estructuras y su intercambio rendimiento/confiabilidad
- Historia y por qué aparece cada invento

Resumen · Los cuatro errores a no repetir

1. **"El SO deja de responder"** ante una instrucción privilegiada → **el hardware la rechaza** antes de ejecutarla.
2. **Los 64 bytes "son" las particiones** → la tabla **describe**, no contiene. 16 bytes por renglón, sea de 50 MB o de 2 TB. Tabla completa de 4 particiones = 64 bytes.
3. **Cliente-servidor para algo sensible a la latencia** → es **el más lento** de todos. Y no confundir la estructura del SO con la arquitectura de una aplicación.
4. **Capas para tolerancia a fallos** → en capas **todo sigue en modo kernel**: un driver que falla tumba todo igual.

Resumen · Las cuatro visualizaciones interactivas

Todas accesibles desde `visuales/index.html`:

- **es_programada_interrupciones_dma.html** — las tres formas de E/S, con línea de tiempo y costo de CPU calculado.

- **arranque_y_sector_de_booteo.html** — el arranque paso a paso; se puede cambiar la partición activa.
- **interrupciones_8086.html** — simulador de INT 10h/16h/17h/21h con pantalla de DOS y detección de los errores típicos.
- **estructuras_de_so.html** — las cinco estructuras y qué pasa en cada una cuando falla un driver.